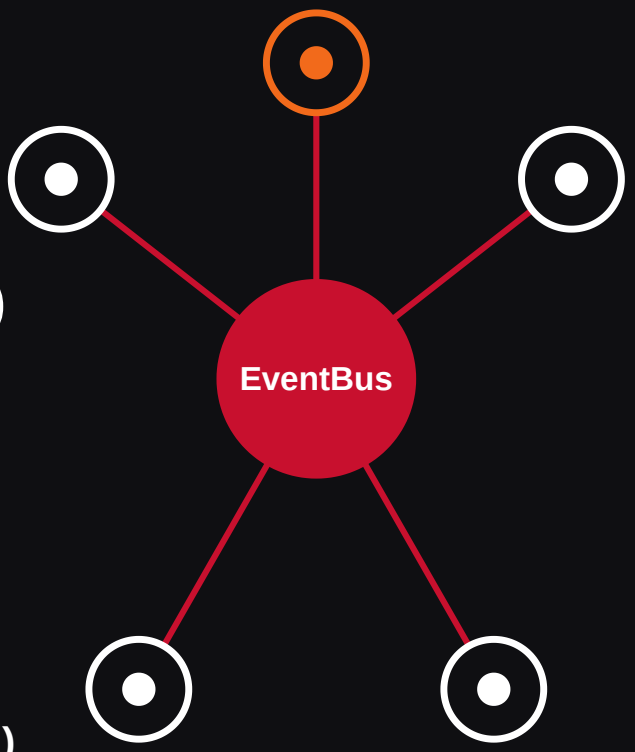
ATIVIDADE 03 · SISTEMAS DISTRIBUÍDOS

Manual do Grupo



Event-Driven Architecture (EDA)

Roteiros, estudo e perguntas para os cinco integrantes · sistema de pedidos em Java

QUEM FALA O QUÊ

Enzo Aquino	explicação do modelo
Caio Abreu	diagramas da arquitetura
Wyllian Mariano	fluxo de dados
Cassiano Abreu	casos de uso no mercado
Gabriel Naoki	demonstração ao vivo

Faculdade SENAI FATESG · Análise e Desenvolvimento de Sistemas · 4º período · Prof. Ramyres · Goiânia, 2026

Sumário

Cada pessoa tem uma parte própria (2 a 6); as partes 0, 1 e 7 e o Apêndice B são leitura de todos.

00 Leia primeiro: como usar este manual 4
0.1 Para que serve e como navegar 4
0.2 A apresentação em um olhar 5
0.3 O que o professor vai avaliar 5
0.4 Decisões em aberto para o grupo 6
0.5 Antes do primeiro ensaio: checklist de consistência 7

01 Base comum: o projeto que todos precisam conhecer 8
1.1 O sistema em um minuto 8
1.2 O elenco: 5 eventos e 5 serviços 8
1.3 Por dentro do EventBus 9
1.4 Números para decorar 11
1.5 Limites conhecidos do projeto 11
1.6 Mapa dos arquivos do repositório 12

02 Enzo Aquino: explicação do modelo 14
2.1 Seu papel e suas fronteiras 14
2.2 Roteiro de fala 14
2.3 Como conduzir os slides 15
2.4 Estudo: o que é EDA 15
2.5 Estudo: os quatro elementos e o broker 16
2.6 Estudo: os cinco padrões do slide 3 17
2.7 Perguntas que o professor pode fazer 18

03 Caio Abreu: diagramas da arquitetura 20
3.1 Seu papel e suas fronteiras 20
3.2 Roteiro de fala 20
3.3 Como conduzir os 3 slides 21
3.4 Estudo: o que é um diagrama de arquitetura em EDA 21
3.5 Slide 4 em profundidade: arquitetura de pastas 22
3.6 Slides 5 e 6 em profundidade 23
3.7 Perguntas que o professor pode fazer 24

04 Wyllian Mariano: fluxo de dados 25
4.1 Seu papel e suas fronteiras 25
4.2 Roteiro de fala 25
4.3 Como conduzir os 2 slides 26
4.4 Estudo: o caminho de um pedido 27
4.5 Estudo: lendo a saída real do cenário 3 27
4.6 Estudo: assincronia, concorrência e consistência eventual 28
4.7 Perguntas que o professor pode fazer 29

05 Cassiano Abreu: casos de uso no mercado 31
5.1 Seu papel e suas fronteiras 31
5.2 Roteiro de fala 31
5.3 Como conduzir os 2 slides 32
5.4 Estudo: os quatro casos e o que foi conferido 32
5.5 Estudo: do projeto à produção 33
5.6 Estudo: a lição da Netflix 34

5.7 Perguntas que o professor pode fazer 34

06 Gabriel Naoki: demonstração ao vivo 36

6.1 Seu papel e suas fronteiras 36
6.2 Preparação e plano B 36
6.3 Roteiro de fala e de execução 37
6.4 Os 8 cenários e as saídas esperadas 38
6.5 O estado entre os cenários 40
6.6 O repositório e as evidências de commits 40
6.7 Perguntas que o professor pode fazer 41

07 Banco de perguntas gerais 43

7.1 Como usar esta parte 43
7.2 Conceituais 43
7.3 Sobre o código 44
7.4 Sobre a apresentação 44

A Apêndices 46

A Glossário 46
B Cola de uma página: frases prontas 47
C Checklist final, por pessoa 47
D Como este manual foi conferido 48

00 Leia primeiro: como usar este manual

PARA TODOS | 10 MIN DE TEORIA + DEMO

0.1 Para que serve e como navegar

Este manual junta, em um só arquivo, o que cada integrante precisa para **apresentar** a sua parte e **responder** ao professor. Cada pessoa tem uma parte própria (2 a 6). As partes 0, 1 e 7 e o Apêndice B são leitura de todos. Assim ninguém estuda a mesma coisa duas vezes e ninguém fala o que é do colega.

A estrutura de cada parte pessoal

Bloco	Para que serve
Papel e fronteiras	O que é seu, o que é do colega e onde a sua fala começa e termina.
Roteiro de fala	O texto para falar, com tempo por slide. É o único trecho que entra na apresentação.
Como conduzir os slides	O que apontar na tela, em que ordem, e o que não ler.
Material de estudo	Aprofundamento para você responder com segurança. Nada disto é falado.
Perguntas do professor	Perguntas prováveis, com resposta curta para dizer em voz alta.

> **PARA DECORAR**
> Roteiro é o que você **fala**. Estudo é o que você **sabe**. Nada do estudo entra na fala.

Etiquetas que aparecem ao longo do manual

> **ATENÇÃO**
> Aviso sobre algo que escapa fácil na hora de falar.

> **CUIDADO**
> Armadilha ou erro comum; leia antes de ensaiar.

> **SE PERGUNTAREM**
> Frase pronta para responder à pergunta mais provável.

> **DECISÃO DO GRUPO**
> Ponto em que o grupo precisa combinar algo entre si.

Quem lê o quê

Quem	Leia primeiro	Sua parte	Antes da banca
Enzo	Parte 0 e Parte 1	Parte 2 · Explicação do modelo	Parte 7, Apêndice B

Caio	Parte 0 e Parte 1	Parte 3 · Diagramas da arquitetura	Parte 7, Apêndice B
Wyllian	Parte 0 e Parte 1	Parte 4 · Fluxo de dados	Parte 7, Apêndice B
Cassiano	Parte 0 e Parte 1	Parte 5 · Casos de uso no mercado	Parte 7, Apêndice B
Naoki	Parte 0 e Parte 1	Parte 6 · Demonstração ao vivo	Parte 7, Apêndice B

Onde cada assunto é explicado (fonte única)

Assunto	Onde está	Dono
O sistema em um minuto, eventos e serviços	1.1 e 1.2	todos
Por dentro do EventBus (pub/sub, threads, retry, Event Store, DLQ)	1.3	todos (Caio implementou)
O que é EDA, broker, os 5 padrões, coreografia	2.4 a 2.6	Enzo
Como ler diagramas, pastas, mapa da arquitetura	3.4 a 3.6	Caio
Caminho do pedido, saga, saída real, concorrência	4.4 a 4.6	Wyllian
Empresas, Kafka x RabbitMQ, da simulação à produção	5.4 a 5.6	Cassiano
Os 8 cenários, preparação da demo, commits	6.2 a 6.6	Naoki
Mapa dos arquivos do repositório	1.6	todos

0.2 A apresentação em um olhar

A parte teórica dura **10 minutos**. O Caio já tinha 2min30 de 10min nos slides 4 a 6; o mesmo tempo para os outros três fecha a conta e atende ao critério de **divisão equilibrada de fala**. A demonstração do Naoki fica fora dos 10 minutos.

Ordem	Slides	Quem	Tema	Tempo	Passa a palavra para
1	1 a 3	Enzo	Explicação do modelo	0:00 – 2:30	Caio
2	4 a 6	Caio	Diagramas da arquitetura	2:30 – 5:00	Wyllian
3	7 e 8	Wyllian	Fluxo de dados	5:00 – 7:30	Cassiano
4	10 e 11	Cassiano	Casos de uso no mercado	7:30 – 10:00	Naoki
5	9 e 12 + terminal	Naoki	Demonstração ao vivo e encerramento	fora dos 10 min	professor

> **CUIDADO**
> Os tempos por pessoa são uma **proposta** baseada no tempo do Caio. Se cada um passar 15 segundos, o grupo estoura 11 minutos. Cronometrem em voz alta e ajustem as passagens.

0.3 O que o professor vai avaliar

Os critérios valem de 0 a 100: **50% apresentação da arquitetura** e **50% implementação e live preview**.

Critério	Peso	Como o grupo atende	Onde está

Domínio teórico do modelo	50% (apresentação)	Enzo explica; qualquer integrante pode responder perguntas, então todos leem as partes 1 e 7	1.3, Parte 2, Parte 7
Clareza dos diagramas e do fluxo de comunicação		Caio apresenta a estrutura (slides 5 e 6); Wyllian apresenta o movimento (slides 7 e 8)	Parte 3, Parte 4
Postura técnica e divisão equilibrada de fala		2:30 para cada um dos quatro; passagens combinadas	0.2, 0.4
Live preview funcionando	50% (implementação)	Naoki roda o sistema; plano B pronto se algo falhar	6.2, 6.3
Aderência do código ao modelo		Cada conceito do slide 3 tem arquivo e cenário correspondentes	1.3, 2.6
Commits de todos os 5 integrantes		Todos têm commits e pull request mesclado; conferir se os 5 aparecem em Contributors	6.6

ATENÇÃO

Qualquer integrante pode ser perguntado sobre qualquer parte. Por isso as partes 1 e 7 e o Apêndice B são leitura de todos. O professor tende a perguntar a quem está mais calado.

0.4 Decisões em aberto para o grupo

Ao cruzar o deck de 12 slides com a divisão de fala, aparecem três pontos que ainda não estão combinados. **Resolvam antes do primeiro ensaio.**

DECISÃO DO GRUPO 1 · ONDE FICA O SLIDE 9 (DEMONSTRAÇÃO)?

No deck ele está entre o Wyllian (slide 8) e o Cassiano (slide 10), e vem rotulado como parte do Naoki, mas a demo ao vivo do Naoki fica fora dos 10 minutos.

Opção A (recomendada pelo manual): mover o slide 9 para depois do slide 11. Cassiano fecha a teoria, passa para o Naoki, que abre a demo no slide 9. Os roteiros deste manual seguem esta opção.

Opção B: manter a ordem. Então o Wyllian diz uma frase de ponte no slide 9 ("agora vamos ver isto funcionando, mas antes o Cassiano mostra o mercado") e a demo acontece depois, com a tela do terminal.

Nota: a Opção A deixa o deck com a ordem 1–8, 10, 11, 9, 12. Se for adotada, renumerem o deck antes do ensaio.

DECISÃO DO GRUPO 2 · QUEM FALA OS SLIDES 1 E 12?

Proposta: o Enzo abre no slide 1 (já está no roteiro dele) e o Naoki encerra no slide 12, "Quem fez o quê?", ao fim da demo, apresentando o repositório.

DECISÃO DO GRUPO 3 · FRASES DE PASSAGEM.

Estão escritas nos roteiros, mas combinam as palavras exatas de cada um. A passagem é onde o grupo mais perde ritmo.

De	Para	Frase sugerida
Enzo	Caio	"Agora o Caio vai mostrar como isso se organiza no projeto e qual é a arquitetura."
Caio	Wyllian	"Agora o Wyllian vai mostrar o caminho de um pedido, passo a passo, e a saída real da execução."
Wyllian	Cassiano	"Agora o Cassiano mostra onde isso é usado no mercado e como seria em produção."

Cassiano	Naoki	"Com isso encerramos a parte teórica. O Naoki vai rodar o sistema ao vivo."

0.5 Antes do primeiro ensaio: checklist de consistência

Conferir estes itens evita que a banca encontre algo que o manual não previu.

Item	Como conferir	Responsável
Slide 7 diz "PedidoService: confirma e notifica"	Corrigir para: "PedidoService: marca CONFIRMADO; NotificacaoService avisa o cliente" (ver §4.2)	Wyllian
Slide 8 usa "→" em "PED-1 → CANCELADO"	No terminal aparece "->". Trocar no slide para ficar igual à saída real	Wyllian
Repositório tem 2 PDFs em `docs/`	Abrir o GitHub; confirmar `Slides EDA.pdf` e `Descricao Tecnica EDA.pdf`	Naoki
Contributors mostra os 5 integrantes	Abrir a página do repositório no GitHub logado como o professor ou em aba anônima	Naoki
Build com Maven funciona na máquina de quem vai rodar	`mvn clean package` e depois `java -jar target/eda-seminario.jar`. Se o Maven não baixar plugins, usar o comando de §6.2 (plano B, `javac`)	Naoki
Saídas dos 8 cenários batem com §6.4	Rodar a demo do início, em sequência, uma vez (ver 6.5 sobre estado)	Naoki

01 Base comum: o projeto que todos precisam conhecer

PARA TODOS LEITURA OBRIGATÓRIA

1.1 O sistema em um minuto

Um sistema de pedidos em **Java 21 puro**, sem dependências externas, em que cinco serviços **nunca se chamam**. Eles publicam eventos em um intermediário, o **EventBus**, e reagem a eventos que lhes interessam. Tudo roda em memória, na mesma JVM, e uma demo no terminal executa 8 cenários.

Item	Detalhe
Linguagem	Java 21 (records e `switch` com seta)
Build	Maven; gera `target/eda-seminario.jar`; classe principal `eda.App`
Dependências	nenhuma: só a biblioteca padrão do Java
Código	11 arquivos `.java`, 475 linhas, nos pacotes `core`, `eventos` e `servicos` (mais `App`)
Execução	menu no terminal com 8 cenários; digite o número e Enter; `0` encerra
Repositório	github.com/GabrielNaokiUT/EventDrivenArchitecture_DesenvolvimentoSistemasDistribuidos

A história de um pedido em cinco frases

1. O cliente faz um pedido: o `PedidoService` cria o `PED-n` com status CRIADO e publica `PedidoCriado`.
2. O `PagamentoService` reage: aprova se o valor for de até R$ 5.000 (publica `PagamentoAprovado`) ou recusa (publica `PagamentoRecusado`).
3. Se aprovou, o `EstoqueService` reage: reserva o item (`EstoqueReservado`) ou avisa que falta (`EstoqueInsuficiente`).
4. Se reservou, o pedido fica CONFIRMADO e o cliente recebe um e-mail. Se algo falhou, o pedido fica CANCELADO, o cliente é avisado e, no caso de falta de estoque, o pagamento é estornado.
5. Do começo ao fim, a `AuditoriaService` registra tudo. **Nenhum serviço chamou outro serviço.**

> **PARA DECORAR**
> A frase da apresentação inteira: **"quem publica um evento não sabe quem vai reagir a ele."** Os cinco devem saber dizê-la sem olhar o papel.

O mapa visual da arquitetura está em 3.6; o caminho do pedido, em 4.4.

1.2 O elenco: 5 eventos e 5 serviços

Fonte: `eventos/Eventos.java`, linhas 10 a 27.

Evento	Campos	Quem publica	Quem consome
`PedidoCriado`	pedidoId, cliente, produto, quantidade, valor	Pedido	Pagamento, Auditoria
`PagamentoAprovado`	pedidoId, cliente, produto, quantidade, valor	Pagamento	Estoque, Auditoria
`PagamentoRecusado`	pedidoId, cliente, motivo	Pagamento	Pedido, Notificação, Auditoria
`EstoqueReservado`	pedidoId, cliente, produto, quantidade	Estoque	Pedido, Notificação, Auditoria
`EstoqueInsuficiente`	pedidoId, cliente, produto, valor	Estoque	Pedido, Pagamento (estorno), Notificação, Auditoria

Todos são `record` (imutáveis) e implementam a interface `Event`. Imutável combina com o conceito: um evento é **um fato que já aconteceu**, e fatos não mudam. Repare que cada evento **carrega os dados de que o consumidor precisa**: o `PagamentoAprovado` leva produto e quantidade, então o Estoque age sem perguntar nada ao Pedido (padrão Event-carried state transfer, seção 2.6).

Os cinco serviços (pasta `servicos/`)

Serviço	O que faz	Regras e números do código
`PedidoService`	Cria pedidos e guarda o status de cada um; atualiza o status quando chegam os eventos de resultado	Ids `PED-1`, `PED-2`…; status CRIADO, CONFIRMADO, CANCELADO (pagamento recusado), CANCELADO (sem estoque). Código: linhas 22–27 (assinaturas) e 30–37 (`criarPedido`)
`PagamentoService`	Aprova ou recusa o pagamento; estorna quando falta estoque	Pausa de 500 ms simula o gateway (linha 23); limite `LIMITE = 5000.0` (linha 12); estorno com pausa de 300 ms (linhas 34–37)
`EstoqueService`	Reserva o item ou avisa que falta	Estoque inicial Notebook 5, Mouse 50, Monitor 1 (linha 17); método `synchronized` (linha 26); idempotente (linhas 27–30); pausa de 400 ms (linha 31)
`NotificacaoService`	Envia o e-mail (simulado) ao cliente	Pausa de 200 ms (linha 30); `simularFalhas(n)` força erros para a demo de retry e DLQ (linhas 25–27)
`AuditoriaService`	Registra todos os eventos	Assina `Event.class` (linha 10); imprime `[AUDIT] Tipo | pedidoId` (linha 11)

1.3 Por dentro do EventBus

O `EventBus` (84 linhas, `core/EventBus.java`) é a peça central e é o que o grupo chama de **broker**. Cada evento publicado percorre o caminho abaixo. Esta é a seção mais importante da base comum: **todo mundo precisa conseguir explicá-la.**

Passo	O que acontece	Onde no código
1 · `publish()`	Chamado pelo produtor; retorna na hora	linhas 44–55
2 · Envelope	Id curto (8 caracteres) + horário + o evento	linha 45

3 · Event Store	Grava o envelope **antes** de entregar	linha 46
4 · Inscritos	Classe exata do evento + quem assina `Event`	linhas 48–50
5 · Pool	`executor.submit()`: 1 tarefa por inscrito, pool de 8 threads	linhas 29, 52–54
6 · `deliver()`	Até 3 tentativas, com pausa de 300 ms entre elas; falhou 3 vezes vai para a DLQ	linhas 57–71

Resumo: o produtor chama `publish()` e **volta na hora**; o bus grava o evento no Event Store e entrega a cada inscrito por meio de um pool de 8 threads. Cada entrega tenta até **3 vezes**; se falhar todas, o evento vai para a **DLQ**, sem atrapalhar os outros consumidores.

Trecho resumido do código (os nomes são os reais; comentários e algumas linhas foram simplificados):

```
public <T extends Event> void subscribe(String subscriber, Class<T> type, Consumer<T> handler) {
    subscriptions.computeIfAbsent(type, k -> new CopyOnWriteArrayList<>())
            .add(new Subscription(subscriber, e -> handler.accept(type.cast(e))));
}

public void publish(Event event) {
    Envelope envelope = new Envelope(UUID.randomUUID().toString().substring(0, 8), Instant.now(), event);
    eventStore.add(envelope);                                   // grava antes de entregar
    List<Subscription> destinos = new ArrayList<>();
    destinos.addAll(subscriptions.getOrDefault(event.getClass(), List.of()));
    destinos.addAll(subscriptions.getOrDefault(Event.class, List.of()));  // quem escuta tudo (auditoria)
    for (Subscription s : destinos) {
        executor.submit(() -> deliver(envelope, s));            // entrega em outra thread
    }
}

private void deliver(Envelope envelope, Subscription s) {
    for (int tentativa = 1; tentativa <= MAX_TENTATIVAS; tentativa++) {   // MAX_TENTATIVAS = 3
        try { s.handler().accept(envelope.event()); return; }
        catch (Exception e) { Log.info("EventBus", "..."); Log.pausa(300); }
    }
    deadLetterQueue.add(envelope);                              // falhou 3 vezes: DLQ
}
```

Os termos da barra do EventBus (slide 5)

Termo	O que significa	No código e na demo
Publicar-assinar (Pub/Sub)	Quem tem interesse se inscreve em um tipo de evento; quem produz só publica; o broker liga as duas partes. O "tópico" é a **classe**.	`subscribe(nome, PagamentoAprovado.class, handler)` inscreve; `publish(evento)` procura os inscritos.
Entrega assíncrona	O `publish` retorna na hora; a entrega ocorre em outras threads, em paralelo, sem o produtor esperar. Efeito: **consistência eventual**: o estado só fica correto depois de um instante.	`Executors.newFixedThreadPool(8)`; threads `worker-N`. Cenário 4: logo após publicar, os 3 pedidos estão CRIADO; depois convergem para CONFIRMADO.
Retentativas (3x)	Se o consumidor lança exceção, o broker tenta de novo, até 3 tentativas no total, com pausa de 300 ms entre elas.	`MAX_TENTATIVAS = 3`. Cenário 5: a notificação falha 2 vezes e funciona na 3ª.
Event Store	Registro de todos os eventos publicados, em ordem, com histórico. Aqui é só leitura: **não há replay**.	Lista `eventStore`, gravada antes da entrega. Cenário 8.

DLQ (Dead Letter Queue)	Para onde vai o evento que falhou em todas as tentativas. Ele não se perde e não trava os outros consumidores.	`deadLetterQueue`. Cenário 6: depois de 3 falhas o evento vai para a DLQ e o pedido continua CONFIRMADO.
Envelope	Embrulho do evento com id curto e horário. Separa dados de controle dos dados de negócio.	`record Envelope(String id, Instant occurredAt, Event event)`

ATENÇÃO

Por que a Auditoria recebe tudo? Porque o `publish` busca **duas** listas de inscritos: a da classe exata do evento e a de `Event.class`. É um tratamento explícito, não herança automática. Assinar uma interface intermediária qualquer não funcionaria.

CUIDADO

Se perguntarem "por que 8 threads?": é um valor escolhido para a demo. Um pool fixo limita o paralelismo e evita criar uma thread por evento. Não há motivo técnico mais profundo no código, então **não invente um**.

1.4 Números para decorar

Número	O que é	Onde
5	eventos e serviços (5 de cada)	`Eventos.java`, pasta `servicos/`
8	threads do pool; também o total de cenários da demo	`EventBus.java` linha 29; `App.java` menu
3	tentativas de entrega antes da DLQ	`EventBus.java` linha 17
300 ms	pausa entre tentativas; também o tempo do estorno	`Log.pausa(300)`; `PagamentoService.java` linha 35
500 ms	tempo do pagamento (simula o gateway)	`PagamentoService.java` linha 23
400 ms	tempo da reserva de estoque	`EstoqueService.java` linha 31
200 ms	tempo do envio de e-mail	`NotificacaoService.java` linha 30
R$ 5.000	limite do pagamento (acima disso é recusado)	`PagamentoService.java` linha 12
5 / 50 / 1	estoque inicial de Notebook / Mouse / Monitor	`EstoqueService.java` linha 17
2:30 de 10:00	tempo de fala de cada apresentador da teoria	seção 0.2

1.5 Limites conhecidos do projeto

O projeto é uma **simulação didática**, e o README e o slide 11 dizem isso. Conhecer os limites é o que dá segurança: se o professor apontar um deles, o grupo já sabia e tem resposta pronta. **Não escondam; antecipem.**

Limite	O que acontece	Resposta curta
Tudo em memória, um só processo	Se a JVM encerra, o Event Store e a DLQ somem; os serviços dividem o mesmo processo	"É uma simulação do broker. Em produção troca-se por Kafka ou RabbitMQ e a lógica dos serviços não muda."
Event Store sem replay	Guarda o histórico, mas não reprocessa eventos	"Serve de histórico. Replay exige um broker com log persistido, como o Kafka."

DLQ só guarda	O evento que falhou 3 vezes fica parado	"Em produção haveria alerta e reprocessamento; está no slide 11."
Entrega "pelo menos uma vez"	O retry pode repetir um efeito; só o Estoque é idempotente	"Aplicamos idempotência onde o efeito é crítico, que é baixar estoque. O cenário 7 prova."
Sem garantia de ordem entre consumidores	Rodam em paralelo; no cenário 4 o PED-3 reservou antes do PED-2	"A ordem dentro de um pedido vem da causalidade: cada evento só nasce depois do anterior."
Retry bloqueia uma thread	`Thread.sleep(300)` roda dentro do pool de 8 threads	"Aceitável na demo. Em produção, retry seria uma fila com atraso."
Estorno é só um registro	O Pagamento não guarda estado nem fala com banco	"Representa a transação de compensação da saga."
Sem testes automatizados	O repositório não tem pasta `test`	"Os 8 cenários da demo funcionam como testes de aceitação."
Pedido não persiste	Status de pedido e estoque vivem em `Map` e `HashMap` na memória; reiniciar zera tudo	"Estado de demo. Em produção, cada serviço tem seu banco."
Entrada não validada	`criarPedido` aceita quantidade ou valor fora do esperado sem checar (ex.: quantidade 0 ou negativa)	"Validação de entrada ficou fora do escopo da demo. Em produção ela entra na borda, antes de publicar."
Sem outbox	Se o `PedidoService` cair entre criar o pedido e publicar, o evento se perde (ver 7.3)	"Não tratamos. Em produção seria o padrão Outbox."

ATENÇÃO · O QUE A DEMO NÃO DEMONSTRA

Para não ser pego de surpresa, o grupo deve **dizer** que a demo não cobre: persistência, autenticação, validação de entrada, testes automatizados, Outbox, e múltiplos processos ou máquinas. Dizer isso antes da banca é melhor do que ser perguntado.

1.6 Mapa dos arquivos do repositório

Ela lista tudo que está no repositório, porque o professor pode abrir qualquer arquivo.

Arquivo ou pasta	O que é	Quem fez
`pom.xml`	Configuração do Maven. Java 21, sem dependências externas. Gera `target/eda-seminario.jar` com `eda.App` como classe principal. Permite `mvn compile exec:java`.	Naoki
`README.md`	O que é EDA, o diagrama em Mermaid, a estrutura de pastas, como executar e a tabela dos 8 cenários.	Naoki
`.gitignore`	Ignora `target/`, `out/`, arquivos de IDE.	Naoki
`docs/Slides EDA.pdf`	Cópia do deck de 12 slides (é o mesmo arquivo entregue ao Classroom).	Naoki
`docs/Descricao Tecnica EDA.pdf`	Descrição técnica em 5 páginas: explica o código, o fluxo, as classes, a demo, as limitações e a participação de cada um. Pode ser usada como segunda fonte para o Cassiano (seção 5.3) e para tirar dúvidas.	Naoki

`src/main/java/eda/App.java`	Menu interativo da demo (8 cenários). Cria o bus e os 5 serviços.	Wyllian
`core/Event.java`, `core/Envelope.java`	Contrato de evento e envelope (id + horário + evento).	Caio
`core/EventBus.java`, `core/Log.java`	Broker (pub/sub, retry, Event Store, DLQ) e log com hora, thread e origem.	Caio
`eventos/Eventos.java`	Os 5 eventos como records dentro de uma classe final com construtor privado.	Enzo
`servicos/PedidoService.java`, `AuditoriaService.java`	Produtor de pedidos e auditoria.	Enzo
`servicos/PagamentoService.java`, `EstoqueService.java`	Pagamento com estorno e estoque idempotente.	Cassiano
`servicos/NotificacaoService.java`	Notificação com falhas simuladas.	Wyllian

> **CUIDADO**
> Não existe pasta `test`. A documentação está no README e nos dois PDFs de `docs/`.

02 Enzo Aquino: explicação do modelo

SLIDES 1 A 3 | 2 MIN 30 DE 10 MIN | DEPOIS: CAIO

2.1 Seu papel e suas fronteiras

Você **abre** a apresentação e entrega o vocabulário que os outros três vão usar: a missão é deixar o público com **uma frase** (serviços não se chamam: publicam eventos e reagem a eles) e **quatro palavras** (evento, produtor, consumidor, broker).

- **É seu:** o que é EDA, a diferença para chamada direta, os quatro elementos e os cinco padrões (slides 1 a 3).
- **Não é seu:** o desenho da arquitetura e as pastas (Caio), a ordem dos passos e a saída no terminal (Wyllian), Kafka e empresas (Cassiano).
- **Atenção:** você é quem o professor mais vai testar em teoria. Estude as seções 2.4 a 2.7 com calma.

2.2 Roteiro de fala

Tempo	Slide	Fala
0:00 a 0:15	SLIDE 1 · ABERTURA	Olá, professor e pessoal. Somos o grupo que vai apresentar a Arquitetura Orientada a Eventos, a EDA. Construímos um sistema de pedidos em Java para mostrar como ela funciona na prática. Eu sou o Enzo e vou explicar o modelo.
0:15 a 1:15	SLIDE 2 · O QUE É EDA?	Pense em uma chamada direta: o Pedido chama o Pagamento e espera. Ele precisa conhecer o Pagamento, e se o Pagamento muda ou cai, o Pedido sente. Na EDA é diferente: os serviços não se chamam. O Pedido apenas publica um evento, avisando que algo aconteceu. Quem tem interesse recebe o evento e reage por conta própria: o Pagamento, o Estoque, a Auditoria, e outros que quisermos somar depois. Isso traz três ganhos: os serviços ficam independentes, novos consumidores entram sem mexer nos outros, e a falha de um não derruba os demais. Por isso a frase que resume tudo é: quem publica não sabe quem vai reagir.
1:15 a 2:20	SLIDE 3 · ELEMENTOS E PADRÕES	Para isso, precisamos de quatro elementos. O evento é um fato que já aconteceu, como PedidoCriado; por isso o nome fica no passado. O produtor publica o evento, como o PedidoService. O consumidor se inscreve e reage; no código, cada serviço faz o subscribe no próprio construtor. E o broker, a classe EventBus, é o intermediário que entrega os eventos; pense nos Correios: você entrega a carta e ele cuida do caminho. Produtor e consumidor nunca se conhecem. O projeto também usa cinco padrões: Publish-Subscribe, que é a base de tudo; Idempotent Receiver, que ignora o evento repetido; Dead Letter Channel, para o evento que falhou; Event-carried state transfer, em que o evento leva os dados de que o consumidor precisa; e a Saga coreografada, que coordena o pedido só por eventos.

2:20 a 2:30	PASSAGEM	Agora o Caio vai mostrar como isso se organiza no projeto e qual é a arquitetura.

Contagem do roteiro: 296 palavras (ritmo calmo: cerca de 125 a 130 palavras por minuto).

ATENÇÃO

Os cinco padrões são citados em **uma frase cada**. Não explique nenhum com profundidade: o professor pode pedir depois, e a seção 2.6 tem a resposta. A Saga coreografada ganha vida quando o Wyllian mostrar o estorno.

2.3 Como conduzir os slides

Slide	O que apontar	Cuidado
1 Capa 0:00 a 0:15	Nada específico. Olhe para o professor, não para a tela.	Seja breve: são 15 segundos.
2 O que é EDA? 0:15 a 1:15	Primeiro o quadro de cima (Chamada direta, Pedido para Pagamento). Depois o quadro laranja: Pedido, EventBus e as três caixas à direita.	Não explique "assíncrono" nem "EventBus por dentro": é assunto do Caio e do Wyllian. Termine na frase da tela.
3 Elementos e padrões 1:15 a 2:20	Os quatro cartões, da esquerda para a direita; depois a barra azul ("Produtor e consumidor não se conhecem"); por fim as cinco pílulas.	Não leia os cartões inteiros nem explique saga. Uma frase por padrão.

SE PERDEREM O FIO

Volte à frase "quem publica não sabe quem vai reagir". Ela fecha qualquer explicação.

2.4 Estudo: o que é EDA

Em uma arquitetura orientada a eventos, os componentes **não se chamam diretamente**: eles trocam **eventos** por meio de um intermediário. Quando algo relevante acontece, o componente publica um evento. Quem se interessou por aquele tipo de evento reage.

Aspecto	Chamada direta	Orientada a eventos
Quem conhece quem	O chamador conhece o serviço chamado	Ninguém conhece ninguém; todos conhecem o broker e o contrato do evento
Espera	O chamador espera a resposta	`publish()` retorna na hora; a reação vem depois
Falha	Falha do chamado afeta o chamador	Falha de um consumidor não derruba os outros (retry e DLQ)
Extensão	Somar um serviço exige mexer no chamador	Basta somar um consumidor (foi assim com a Auditoria)
Visão do processo	Fácil: o código mostra a sequência	Difícil: o fluxo está espalhado em vários consumidores; daí a importância de diagramas e logs
Consistência	Imediata	Eventual: o estado converge depois de um instante

Vantagens e custos

- **Desacoplamento:** serviços evoluem e falham de forma independente.
- **Extensibilidade:** novos consumidores entram sem alterar quem já existe.
- **Resiliência:** o retry e a DLQ isolam a falha de um consumidor.
- **Custo 1, consistência eventual:** logo depois de publicar, o estado ainda não convergiu (cenário 4).
- **Custo 2, depuração:** o caminho de um pedido não está em um único método; é preciso rastrear por logs e diagramas.
- **Custo 3, duplicidade e ordem:** eventos podem se repetir e chegar fora de ordem; os consumidores precisam estar preparados (idempotência).

Evento, comando e mensagem

Conceito	Significado	Exemplo
Evento	Um fato que já aconteceu. Tem nome no passado. Quem publica não espera nada; vários podem reagir.	`PedidoCriado`, `EstoqueReservado`
Comando	Uma ordem para alguém fazer algo. Tem um destinatário certo.	`CriarPedido`, `AprovarPagamento` (o projeto **não** usa)
Mensagem	Termo genérico: qualquer coisa que trafega por um canal. Eventos e comandos são tipos de mensagem.	-

> **SE PERGUNTAREM: QUANDO NÃO USAR EDA?**
> Quando a resposta precisa ser imediata e consistente (um saldo bancário consultado na hora), em sistemas pequenos com poucos componentes, ou quando o time não consegue operar um broker. EDA troca simplicidade por flexibilidade.

2.5 Estudo: os quatro elementos e o broker

Elemento	O que é	No projeto
Evento	Um fato que já aconteceu; imutável	5 `record` em `eventos/Eventos.java`
Produtor	Quem publica o evento	Ex.: `PedidoService.criarPedido` chama `bus.publish(new PedidoCriado(...))`
Consumidor	Quem se inscreve e reage	`bus.subscribe(...)` no construtor de cada serviço
Broker	O intermediário que entrega os eventos	A classe `EventBus` (`core/EventBus.java`)

O que é um broker

Broker significa intermediário, como um corretor. Em sistemas distribuídos, é o componente que fica no meio da comunicação: recebe as mensagens de quem envia e entrega a quem precisa receber, sem que os dois se conheçam. Pense nos **Correios**: você deixa a carta na agência e ela cuida da entrega. Você não sabe quem vai levar nem por qual caminho, e o destinatário não sabe quem enviou até abrir a carta.

Pergunta	Resposta

Broker e Event Bus são a mesma coisa?	Sim. **Event Bus** é o nome da ideia (um canal por onde passam eventos). **Broker** é o papel que ele cumpre (intermediário). No projeto, os dois são a classe `EventBus`.
O que o broker faz no projeto?	Recebe cada evento em `publish()`, guarda no Event Store, descobre quem se inscreveu e entrega em threads, com retentativas e DLQ (veja 1.3).
Por que usar um?	Para desacoplar: quem publica não precisa conhecer, esperar nem saber quantos consumidores existem.
Quais brokers existem no mercado?	Kafka e RabbitMQ são os mais citados e são os que o slide 11 diz que substituiriam o `EventBus` em produção (veja 5.5).
Qual o risco de ter um broker?	Ele vira **ponto único de falha**: se cair, ninguém se comunica. Por isso, em produção, é replicado e persistente.

> **SE PERGUNTAREM**
> Frase pronta: "O broker é o intermediário que recebe os eventos de quem publica e entrega para quem se inscreveu. No nosso código é o EventBus."

2.6 Estudo: os cinco padrões do slide 3

Para cada padrão: o que é, onde está no código e em que cenário da demo ele aparece.

Padrão	O que é	Onde está no projeto	Cenário
Publish-Subscribe	Quem publica não conhece quem assina. O broker conecta os dois pelo tipo do evento.	`EventBus.subscribe()` e `publish()`. É a base de todo o projeto.	todos
Idempotent Receiver	Receptor idempotente: receber o mesmo evento duas vezes tem o mesmo efeito que receber uma. Necessário porque entregas podem se repetir.	`EstoqueService.reservar()` guarda os pedidos já tratados no conjunto `processados` (`EstoqueService.java` linhas 27–30).	7
Dead Letter Channel	Canal para onde vai a mensagem que não conseguiu ser entregue depois das tentativas.	A `deadLetterQueue` do `EventBus`.	6
Event-carried state transfer	O evento carrega os dados que o consumidor precisa, então ele não consulta o serviço de origem.	`PagamentoAprovado` leva cliente, produto, quantidade e valor; o Estoque age sem perguntar ao Pedido.	1, 3
Saga coreografada	Processo longo dividido em passos locais, coordenados só por eventos, sem um maestro. Se um passo falha, uma compensação desfaz os anteriores.	Pedido, Pagamento e Estoque. Se falta estoque, o Pagamento estorna.	3

Trecho que prova a idempotência (`EstoqueService.java`, linhas 26–31):

```
private synchronized void reservar(PagamentoAprovado e) {
    if (!processados.add(e.pedidoId())) {           // add() devolve false se já estava no conjunto
        Log.info("EstoqueService", e.pedidoId() + " evento DUPLICADO ignorado (idempotencia)");
        return;                                      // não baixa o estoque duas vezes
    }
    Log.pausa(400);
    ...
}
```

> **CUIDADO: COREOGRAFADA X ORQUESTRADA**
> Na coreografia, cada serviço reage aos eventos dos outros, sem coordenador: é o nosso projeto. Na orquestração, existe um serviço central que manda cada passo (um "PedidoOrquestrador" chamaria Pagamento e depois Estoque). A orquestração é melhor quando o processo precisa ser fácil de observar (veja 5.6).

Origem dos nomes, se perguntarem: Publish-Subscribe Channel, Idempotent Receiver e Dead Letter Channel vêm do catálogo *Enterprise Integration Patterns* (Hohpe e Woolf). Event-carried State Transfer é descrito por Martin Fowler. A ideia de saga é de Garcia-Molina e Salem (1987); a variante coreografada se popularizou com os microsserviços.

2.7 Perguntas que o professor pode faze

? Qual a diferença entre EDA e Pub/Sub?

EDA é o **estilo arquitetural**, centrado em reagir a eventos. Pub/Sub é o **mecanismo** de publicar e se inscrever que usamos para implementá-la.

? Por que o nome do evento está no passado?

Porque evento é **um fato que já aconteceu** (`PedidoCriado`), não uma ordem. Quem publica comunica o que houve e não exige nada de ninguém; um comando, ao contrário, teria destinatário e viria no imperativo.

? Se produtor e consumidor "não se conhecem", então não há acoplamento?

Há, mas é **muito menor**. Os dois dependem do **contrato do evento** (nome e campos). Se mudarmos os campos de `PagamentoAprovado`, Estoque e Auditoria sentem. O que desapareceu é a dependência direta entre serviços: nenhum importa o outro.

? Quando não usar EDA?

Quando a resposta precisa ser imediata e consistente, em sistemas pequenos ou quando o time não consegue operar um broker. EDA traz simplicidade por flexibilidade e consistência eventual.

? EDA é a mesma coisa que microsserviços?

Não. EDA é um **estilo de comunicação**; microsserviços são um **estilo de decomposição**. Os microsserviços costumam usar EDA, mas dá para usar EDA dentro de um único processo, como no nosso projeto.

? O Event Store permite replay?

Não. O Event Store é só histórico de leitura: guarda o que aconteceu, mas não reenvia nada. Replay exigiria um broker com log persistido (Kafka).

? O que o projeto ganha com o Pagamento e o Estoque não se conhecerem?

Podemos trocar o Estoque (ex.: outro sistema de armazém) sem mexer no Pagamento, desde que o evento `PagamentoAprovado` continue igual. Esse é o ganho de desacoplamento que o slide 2 mostra.

03 Caio Abreu: diagramas da arquitetura

SLIDES 4 A 6 | 2 MIN 30 DE 10 MIN | ANTES: ENZO · DEPOIS: WYLLIAN

3.1 Seu papel e suas fronteiras

Você mostra a **estrutura**: quem existe e quem se conecta a quem. O Wyllian mostra o **movimento**: o que acontece e em que ordem. Mantenha essa divisão e você não invade a fala dele.

- **É seu:** as pastas (slide 4), o diagrama de arquitetura (slide 5) e o diagrama do fluxo de eventos (slide 6).
- **Não é seu:** o que é um broker e os cinco padrões (Enzo; você só cita), o passo a passo do pedido e a saída do terminal (Wyllian).
- **Mas só você pode responder fundo:** você implementou o `EventBus`. As perguntas sobre threads, retry, DLQ e Event Store vão cair em você; domine a seção 1.3.

> **PARA DECORAR**
> O diagrama de arquitetura mostra a **estrutura**: quem existe e quem se conecta a quem. O fluxo de dados do Wyllian mostra o **movimento**: o que acontece e em que ordem.

Slides 4, 5 e 6 (no deck, respectivamente: pastas, arquitetura de serviços e EventBus, fluxo de eventos).

3.2 Roteiro de fala

Tempo	Slide	Fala
0:00 a 0:30	SLIDE 4 · PASTAS	Obrigado, Enzo. Agora eu vou mostrar como a EDA está organizada no nosso projeto, começando pelas pastas. Em core fica o coração do sistema: o contrato Event, o Envelope e o EventBus, que é o broker. Em eventos ficam os records dos eventos, e em servicos ficam os cinco serviços do sistema de pedidos. Em detalhe importante: nenhum serviço importa outro. Todos só conhecem o EventBus.
0:30 a 1:45	SLIDE 5 · ARQUITETURA	Este é o diagrama de arquitetura, o mapa de quem são as peças e como elas se conectam. No centro está o EventBus: publicar e assinar, entrega assíncrona em 8 threads, até 3 tentativas, Event Store e DLQ. Em cima ficam três serviços que publicam e também consomem. O Pagamento reage a ele e publica aprovado ou recusado. O Estoque reage ao pagamento aprovado e publica reservado ou insuficiente. Embaixo ficam dois serviços que só consomem: a Notificação, que avisa o cliente, e a Auditoria, que assina o tipo Event e por isso recebe todos os eventos. O ponto mais importante é o que não existe aqui: nenhuma seta liga um serviço a outro. Tudo passa pelo EventBus. Isso dá desacoplamento, porque quem publica não sabe quem reage. Dá assincronia, porque o publish retorna na hora. E dá extensibilidade: dá para adicionar a Auditoria sem alterar nenhum outro serviço.

1:45 a 2:20	SLIDE 6 · FLUXO DE EVENTOS	O próximo slide mostra o mesmo sistema, agora pelo caminho de cada evento. As setas que saem do EventBus levam o evento a quem se inscreveu, e as que voltam são os novos eventos publicados. O "resultado final" que aparece em duas setas é o conjunto de EstoqueReservado, PagamentoRecusado e EstoqueInsuficiente, que o Pedido e a Notificação escutam.
2:20 a 2:30	PASSAGEM	Agora o Wyllian vai mostrar o caminho de um pedido, passo a passo, e a saída real da execução.

Contagem do roteiro: 297 palavras (ritmo calmo: cerca de 125 a 130 palavras por minuto).

ATENÇÃO

O deck **não tem** um slide com as palavras "Desacoplamento", "Assíncrono" e "Extensível". Esse trecho é só falado, em cima do slide 5. Decore as três palavras e a frase "nenhuma seta liga um serviço a outro", porque nada na tela vai lembrar você.

3.3 Como conduzir os 3 slides

Slide	O que apontar	Cuidado
4 Pastas 0:00 a 0:30	Só as três pastas: `core`, depois `eventos`, depois `servicos`.	Não leia a árvore inteira. Fale só o que o roteiro diz.
5 Arquitetura 0:30 a 1:45	Siga uma ordem fixa: 1) a barra do EventBus, 2) os três serviços de cima, 3) os dois de baixo.	O texto das caixas é pequeno. Não leia: cite os nomes e aponte. A frase "nenhuma seta liga um serviço a outro" não está no slide.
6 Fluxo de eventos 1:45 a 2:20	Percorra só o miolo: Pedido, EventBus, Pagamento, EventBus, Estoque. Depois mostre Notificação e Auditoria.	Não explique seta por seta: isso é a parte do Wyllian (slides 7 e 8). As setas se cruzam, então aponte com o cursor.

CUIDADO

Este roteiro segue o deck de 12 slides. Se algum colega mudar a ordem ou o conteúdo dos slides 4, 5 e 6, ajuste o roteiro da seção 3.2.

3.4 Estudo: o que é um diagrama de arquitetura em EDA

Um diagrama de arquitetura é um **mapa visto de cima** do sistema. Ele mostra as grandes peças e como elas se comunicam. Não mostra o código nem cada passo da lógica: mostra **quem participa e por onde a informação circula**. Em EDA, ele responde três perguntas:

01 · Quem gera os eventos?	02 · Por onde eles passam?	03 · Quem reage a eles?
Os serviços que **publicam**, avisando que algo aconteceu.	Pelo **EventBus**, o canal central que entrega tudo.	Os serviços que **se inscrevem** e agem sozinhos.

O que significam as setas

Em EDA, uma seta mostra o **caminho de um evento** (publicação ou entrega). Ela **não** é uma chamada de método em que alguém espera a resposta. Por isso, nos slides, todas as setas entram ou saem do EventBus.

Como ele se diferencia de outros diagramas

Diagrama	O que mostra
Fluxograma	Passos e decisões de um algoritmo.
Classes (UML)	Classes do código e as relações entre elas.
Arquitetura	Componentes e canais que os ligam. É o seu **slide 5**.
Fluxo de eventos	O mesmo sistema, desenhado pelo caminho de cada evento. É o seu **slide 6**.
Fluxo de dados / sequência	A ordem em que as coisas acontecem. São os slides 7 e 8 do Wyllian.

3.5 Slide 4 em profundidade: arquitetura de pastas

O roteiro cita só as três pastas. Aqui está o resto, arquivo por arquivo, na ordem da árvore do slide.

Item	O que é e o que faz
`pom.xml`	Configuração do Maven. Java 21, sem dependências externas. Gera `target/eda-seminario.jar` com `eda.App` como classe principal; permite `mvn compile exec:java`.
`README.md`	Documentação: o que é EDA, diagrama em Mermaid, estrutura de pastas, como executar e a tabela dos 8 cenários.
`docs/`	Contém **dois PDFs**: `Slides EDA.pdf` (o deck de 12 slides) e `Descricao Tecnica EDA.pdf` (descrição técnica de 5 páginas).
`App.java`	Menu interativo da demo (parte do Wyllian). Cria o `EventBus` e os 5 serviços, lê a opção digitada e executa um dos 8 cenários.
`core/Event.java`	Interface: o contrato de todo evento. Exige `pedidoId()` e fornece `type()`, que devolve o nome da classe (ex.: `PedidoCriado`).
`core/Envelope.java`	Record com id curto, horário (`Instant`) e o evento. É o "envelope" que embrulha o evento ao publicar.
`core/EventBus.java`	O broker: `subscribe`, `publish`, entrega em threads, retry, Event Store e DLQ. É a parte que você implementou.
`core/Log.java`	Utilitário de log no formato `hora [thread] origem mensagem`. Também tem `pausa(ms)`, que simula demora. É o que aparece na tabela do slide 8.
`eventos/Eventos.java`	Os 5 eventos como records imutáveis, agrupados em uma classe `final` com construtor privado.
`servicos/`	Os 5 serviços. Cada um recebe o `EventBus` no construtor e já se inscreve nos eventos que lhe interessam.

PONTO FORTE

Direção das dependências: `servicos` depende de `eventos` e de `core`; `eventos` depende só de `core`; o `core` não conhece nada do negócio. Por isso o broker poderia ser reaproveitado em outro sistema, e os serviços nunca se importam entre si. (Conferido no código: nenhum arquivo de `servicos/` importa outro serviço.)

3.6 Slides 5 e 6 em profundidade

A frase que resume tudo: **quem publica um evento não sabe quem reage a ele.** Os serviços só conversam por meio do EventBus. Esta é uma réplica do seu slide 5, para estudar a leitura.

Serviço	Publica	Consome
PedidoService	PedidoCriado	EstoqueReservado, PagamentoRecusado, EstoqueInsuficiente
PagamentoService	PagamentoAprovado, PagamentoRecusado	PedidoCriado, EstoqueInsuficiente
EstoqueService	EstoqueReservado, EstoqueInsuficiente	PagamentoAprovado
NotificacaoService	nada	EstoqueReservado, PagamentoRecusado, EstoqueInsuficiente
AuditoriaService	nada	todos os eventos (assinatura do tipo `Event`)

A ordem de leitura do slide 5:

1. **A barra preta (EventBus):** o centro de tudo; as cinco características estão escritas nela.
2. **Os três serviços de cima:** publicam e consomem. Siga a cadeia: Pedido publica, Pagamento reage, Estoque reage.
3. **Os dois serviços de baixo:** só consomem. Notificação avisa o cliente; Auditoria registra tudo.
4. **O que não existe:** nenhuma seta entre serviços. Fale isso por último, com pausa.

As três propriedades e a prova no código

Propriedade	O que significa	Como provar
Desacoplamento	Quem publica não sabe quem reage.	Nenhum serviço importa outro; todos só recebem o `EventBus`.
Assincronia	O `publish` retorna na hora; a reação vem depois.	`executor.submit(...)` no `publish`. Cenário 4: status CRIADO / CRIADO / CRIADO logo após publicar.
Extensibilidade	Dá para somar um consumidor sem mexer nos outros.	A `AuditoriaService` tem 13 linhas e é uma única chamada `subscribe`. Veja um serviço novo em 6.7.

O slide 6: o mesmo sistema, pelo caminho de cada evento

No slide, "resultado final" são os eventos `EstoqueReservado`, `PagamentoRecusado` e `EstoqueInsuficiente`: os que encerram o processo e que Pedido e Notificação escutam.

Evento	Quem publica	Quem consome
`PedidoCriado`	PedidoService	PagamentoService
`PagamentoAprovado`	PagamentoService	EstoqueService
`PagamentoRecusado`	PagamentoService	PedidoService, NotificacaoService
`EstoqueReservado`	EstoqueService	PedidoService, NotificacaoService
`EstoqueInsuficiente`	EstoqueService	PedidoService, PagamentoService (estorno), NotificacaoService

todos os eventos	qualquer serviço	AuditoriaService

ATENÇÃO

No slide 6, repare que o `PagamentoService` **também recebe** `EstoqueInsuficiente`: é a seta que faz o estorno. Se alguém perguntar "por que o Pagamento escuta o Estoque?", a resposta é a compensação da saga, e o Wyllian a mostra no slide 8.

3.7 Perguntas que o professor pode fazer

? O que acontece se o broker cair?

O `EventBus` daqui roda em memória, na mesma JVM: se o processo cai, os eventos se perdem. Em produção usa Kafka ou RabbitMQ, replicados e com persistência (slide 11).

? E se um consumidor falhar?

O `EventBus` tenta entregar até 3 vezes. Se continuar falhando, o evento vai para a Dead Letter Queue e os outros serviços seguem normalmente (cenário 6 da demo).

? Por que dois diagramas, nos slides 5 e 6?

O slide 5 mostra a **estrutura**: quem publica e quem consome. O slide 6 mostra o **caminho de cada evento**. É o mesmo sistema visto de dois ângulos.

? Se nenhuma seta liga um serviço a outro, como o Pagamento sabe que existe um PedidoCriado?

Ele **se inscreve** no construtor: `bus.subscribe("PagamentoService", PedidoCriado.class, this::processar)`. O bus guarda um mapa de classe de evento para lista de inscritos e, ao publicar, entrega a cada um. O serviço conhece o **tipo do evento**, não quem o publica.

? Por que a Auditoria recebe todos os eventos?

Porque assina `Event.class`, a base de todos os eventos, e o `publish` procura os inscritos da classe exata **e** os de `Event.class` (veja 1.3).

? Por que os cinco eventos estão em um único arquivo?

São `record` aninhados em uma classe final com construtor privado (`Eventos`). É só organização: evita cinco arquivos minúsculos, e cada um continua tendo sua própria assinatura no bus.

? Para que serve o Event Store, se não há replay?

Porque o histórico é útil para auditoria e para o cenário 8 mostrar o que foi publicado. Sem replay, o Event Store é só leitura. Dizer isso antes evita que o professor pergunte.

04 Wyllian Mariano: fluxo de dados

SLIDES 7 E 8 | 2 MIN 30 DE 10 MIN | ANTES: CAIO · DEPOIS: CASSIANO

4.1 Seu papel e suas fronteiras

Você mostra o **movimento**: o que acontece com um pedido e em que ordem. O Caio acabou de mostrar a estrutura; você faz o sistema **andar**. É a parte em que a plateia enxerga a EDA funcionando.

- **É seu:** o caminho normal e as duas falhas (slide 7) e a leitura da saída real do cenário 3 (slide 8).
- **Não é seu:** o desenho da arquitetura (Caio), o mercado (Cassiano) e a execução ao vivo (Naoki). Se o slide 9 ficar com você, veja a decisão 1.
- **Atenção:** você escreveu a `NotificacaoService` e o `App`. O professor pode perguntar como as falhas são simuladas e como o menu funciona; os detalhes estão em 4.7.

4.2 Roteiro de fala

Tempo	Slide	Fala
0:00 a 0:10	RECEBE A PALAVRA	Obrigado, Caio. O Caio mostrou quem são as peças. Agora vou mostrar o movimento: o que acontece com um pedido, e em que ordem.
0:10 a 1:10	SLIDE 7 · FLUXO DE UM PEDIDO	No caminho normal, o PedidoService cria o pedido com status CRIADO e publica PedidoCriado. O PagamentoService reage, leva 500 milissegundos para simular o gateway, aprova e publica PagamentoAprovado. O EstoqueService reage a esse evento, leva 400 milissegundos, reserva o item e publica EstoqueReservado. O PedidoService recebe e marca o pedido como CONFIRMADO, e a Notificação avisa o cliente. Agora, com falha: se o valor passa de cinco mil reais, o pagamento é recusado e o pedido é CANCELADO. Se falta estoque, o pedido é cancelado e o pagamento é estornado. Esse estorno é a compensação da saga. Repare que em nenhum momento um serviço chama o outro: cada seta é um evento passando pelo EventBus.

1:10 a 2:20	SLIDE 8 · SAÍDA REAL DO CENÁRIO 3	Esta é a saída real do cenário 3, sem estoque. Cada linha mostra a hora, a thread e o serviço que reagiu. A Carla pediu três monitores, mas existe só um. A thread main apenas publica o PedidoCriado; todo o resto roda em threads worker, em paralelo. O pagamento é aprovado, o Estoque vê que não tem monitor suficiente e publica insuficiente. Esse único evento dispara três reações: o Pedido cancela, a Notificação avisa a Carla e o Pagamento faz o estorno, enquanto a Auditoria registra cada passo. Repare na linha destacada: o estorno vem depois do cancelamento, e ninguém mandou nele. Cada serviço reagiu sozinho. Isso é a saga coreografada. A ordem entre as reações não é garantida: aqui o estorno veio por último porque leva 300 milissegundos. Resumindo: um fluxo inteiro, com falha e compensação, sem nenhuma chamada direta entre serviços.
2:20 a 2:30	PASSAGEM	Agora o Cassiano mostra onde isso é usado no mercado e como seria em produção.

Contagem do roteiro: 297 palavras (ritmo calmo: cerca de 125 a 130 palavras por minuto).

ATENÇÃO · SLIDE 7

O slide 7 diz **"PedidoService: confirma e notifica"**. Quem notifica é a `NotificacaoService`; o `PedidoService` só marca CONFIRMADO. Corrijam o slide para **"PedidoService: marca CONFIRMADO"** e, se quiserem, adicionem "NotificacaoService: avisa o cliente". Assim a fala e o slide dizem a mesma coisa.

CUIDADO · SLIDE 8

No slide, a transição aparece como "PED-1 → CANCELADO (sem estoque)". No terminal a seta é `->` (ver `PedidoService.java` linha 45). Use a mesma forma na tela, porque o slide diz que é a saída real.

DECISÃO DO GRUPO 1 · SE O SLIDE 9 FICAR ENTRE O WYLLIAN E O CASSIANO

Este roteiro supõe a Opção A (ver 0.4): o slide 9 vai para depois do slide 11, e o Naoki abre a demo. Se o grupo escolher a Opção B, acrescente, antes da passagem: "Em seguida vamos ver isso funcionando, mas antes o Cassiano mostra o mercado."

4.3 Como conduzir os 2 slides

Slide	O que apontar	Cuidado
7 Fluxo de um pedido 0:10 a 1:10	Siga as setas laranja da esquerda para a direita: Pedido, Pagamento, Estoque, Pedido. Depois desça para "Com falha" e aponte as duas caixas.	Não volte à estrutura (é do Caio). Fale "CONFIRMADO" e "CANCELADO" olhando a tela; são as palavras que a plateia vai ver na demo.
8 Saída real do cenário 3 1:10 a 2:20	Primeiro o subtítulo ("a thread main publica; o resto roda em threads worker"). Depois a coluna Thread: `main` na primeira linha, `worker-N` no resto. Pare na linha rosa (o estorno).	Não leia as 9 linhas. Use três: a 1 (main), a 5 (SEM ESTOQUE) e a 9 (ESTORNO). Os horários são só para mostrar paralelismo.

DICA

Use o dedo (ou o cursor) para descer pela coluna Thread: a plateia percebe sozinha que só a primeira linha é `main`. Isso vale mais que qualquer explicação sobre assincronia.

4.4 Estudo: o caminho de um pedido

O slide 7 deste desenho mostra um **caminho normal** e dois **desvios de falha**. Cada seta é um evento que passa pelo EventBus; nenhuma é chamada direta.

Caminho	Passos (evento que passa pelo EventBus)
Normal	PedidoCriado → PagamentoAprovado → EstoqueReservado → Pedido CONFIRMADO + e-mail
Falha 1: pagamento recusado	PedidoCriado → PagamentoRecusado → Pedido CANCELADO + e-mail ao cliente
Falha 2: sem estoque	PedidoCriado → PagamentoAprovado → EstoqueInsuficiente → Pedido CANCELADO + Notificação + estorno do Pagamento (3 serviços reagem ao mesmo evento)

Os estados do pedido

Status (como aparece no terminal)	Quando acontece	Quem escreve
`CRIADO`	Assim que o pedido nasce, antes de qualquer reação	`PedidoService.criarPedido`
`CONFIRMADO`	Ao receber `EstoqueReservado`	`PedidoService`
`CANCELADO (pagamento recusado)`	Ao receber `PagamentoRecusado` (valor acima de R$ 5.000)	`PedidoService`
`CANCELADO (sem estoque)`	Ao receber `EstoqueInsuficiente`	`PedidoService`

O slide 7 resume os dois cancelamentos como "CANCELADO"; no terminal aparece o motivo entre parênteses.

Saga e compensação, sem enrolar

Uma **saga** divide um processo longo em **passos locais**; cada serviço faz o seu e publica o resultado. Como não há uma transação única cobrindo tudo, se um passo falha é preciso **desfazer** os anteriores com uma **transação de compensação**. No projeto, o pagamento é o passo que precisa ser desfeito quando o estoque falha: é o **estorno**. Se o pagamento é recusado, nada foi cobrado, então não há o que estornar (e, de fato, só `EstoqueInsuficiente` dispara o estorno no código).

> **CUIDADO**
> O "estorno" do projeto é apenas uma linha de log com o valor (`PED-1 ESTORNO de R$ 3000.00`). Não há banco nem saldo. Ele representa o **padrão**, e é assim que deve ser apresentado: "a compensação da saga".

4.5 Estudo: lendo a saída real do cenário 3

A linha do tempo abaixo é o slide 8 desenhado. Cada barra é um `Log.pausa()` do código: o sistema não está "lento", está **simulando trabalho** (gateway de pagamento, reserva, e-mail, estorno). Os números dos círculos são as linhas da tabela.

#	Hora	Thread	Serviço	O que aconteceu e por quê
1	04:24:00.024	main	Pedido	Cria PED-1 (CRIADO) e chama `publish`. É a **única** linha na thread main: o publish volta na hora.
2	00.209	worker-2	Auditoria	Primeira a reagir a `PedidoCriado` (assina todos os eventos).

3	00.704	worker-1	Pagamento	Aprovado cerca de 500 ms depois: a pausa que simula o gateway (`Log.pausa(500)`).
4	00.706	worker-4	Auditoria	Registra `PagamentoAprovado`, 2 ms depois de ele ser publicado.
5	01.125	worker-3	Estoque	Cerca de 400 ms depois (`Log.pausa(400)`): pediu 3, tem 1. Publica `EstoqueInsuficiente`.
6	01.132	worker-5	Pedido	Reage ao `EstoqueInsuficiente`: CANCELADO. Sem pausa, por isso quase instantâneo.
7	01.132	worker-8	Auditoria	Registra o mesmo evento, em paralelo ao Pedido.
8	01.336	worker-6	Notificação	E-mail 200 ms depois (`Log.pausa(200)`).
9	01.433	worker-7	Pagamento	Estorno de R$ 3000.00, 300 ms depois (`Log.pausa(300)`). Por isso é o último.

> **SE PERGUNTAREM: POR QUE O ESTORNO VEM DEPOIS DO CANCELAMENTO?**
>
> Porque o cancelamento não tem pausa e o estorno tem 300 ms. A **ordem entre reações independentes não é garantida**: com outras pausas, o estorno poderia aparecer antes do e-mail. O que é garantido é a **causalidade**: ambos só acontecem depois de `EstoqueInsuficiente`.

Leia `worker-N` como "qualquer thread livre do pool": o número não tem significado de negócio. O formato da linha é `hora [thread] serviço mensagem`, definido em `Log.info`.

4.6 Estudo: assincronia, concorrência e consistência eventual

Quem roda onde

A thread **main** executa o `App`: cria o pedido e chama `publish`. Cada **reação** (consumir um evento) roda em uma thread **worker** do pool de 8. Os consumidores de um mesmo evento rodam **em paralelo**, e é por isso que o log mistura serviços.

Consistência eventual, no cenário 4

O cenário 4 cria 3 pedidos seguidos. A linha `>>` do `App` mostra o estado **logo depois** do `publish`, e outra linha mostra o estado final:

```
>> Os 3 publish() ja retornaram. Status agora: CRIADO / CRIADO / CRIADO  (consistencia eventual)
   ... (cerca de 2 segundos de processamento em paralelo)
>> Status final: CONFIRMADO / CONFIRMADO / CONFIRMADO
```

Consistência eventual é isto: por um instante o sistema está "desatualizado", mas **converge** para o estado correto sem ninguém intervir.

Concorrência controlada: `synchronized` do Estoque

Vários pagamentos aprovados podem chegar ao Estoque ao mesmo tempo. O método `reservar` é `synchronized` para que duas threads não leiam o mesmo saldo e baixem o efeito (a pausa de 400 ms ocorre **dentro** do bloqueio, então as reservas saem uma de cada vez):

Etapa	Horários observados (cenário 4)	Leitura
Pagamentos	10.477, 10.480 e 10.481	Em **paralelo**: o Pagamento não tem bloqueio.
Reservas de estoque	10.890 (PED-1), 11.292 (PED-3), 11.694 (PED-2)	Uma a cada ~400 ms: o `synchronized` **serializa**. Note que a ordem foi 1, 3, 2: quem pegou o primeiro saiu primeiro.

Os horários mudam a cada execução; o que se repete é o padrão (paralelo, depois fila) e o fato de a ordem das reservas **não** seguir a criação dos pedidos. Na execução de referência, as reservas também ficaram espaçadas em ~400 ms.

Estruturas seguras para várias threads

Onde	Estrutura	Por que
`PedidoService`	`ConcurrentHashMap`, `AtomicInteger`	Vários workers atualizam o status; a numeração `PED-n` não pode repetir.
`EventBus`	`ConcurrentHashMap`, `CopyOnWriteArrayList`	Inscrições e listas são lidas por várias threads enquanto se publica.
`EstoqueService`	`synchronized` + `HashMap` + `HashSet`	A verificação e a baixa do saldo precisam ser um passo só.
`NotificacaoService`	`AtomicInteger`	Contador de falhas simuladas, decrementado por qualquer thread.

4.7 Perguntas que o professor pode fazer

? Por que o estorno acontece depois do cancelamento? Isso é garantido?

Não é garantido. O cancelamento não tem pausa, então o estorno leva 300 ms e aparece por último. As reações a um mesmo evento são **independentes e paralelas**; só a causalidade é garantida (nada acontece antes de `EstoqueInsuficiente`).

? Se o pagamento for recusado, há estorno?

Não. Nada foi cobrado, então não há o que compensar. Só `EstoqueInsuficiente` dispara o estorno, porque aí o pagamento já tinha sido aprovado.

? O que é consistência eventual? Mostre no projeto.

É o estado ficar correto **depois de um instante**, não na hora. No cenário 4, logo após publicar os 3 pedidos o status é `CRIADO / CRIADO / CRIADO`; depois de alguns segundos converge para `CONFIRMADO`.

? Por que aparecem threads `worker` diferentes?

Porque a entrega usa um pool de 8 threads. Cada tarefa vai para uma thread livre; por isso o número varia. A thread `main` só publica.

? E se dois pedidos disputarem o último item?

O método `reservar` é `synchronized`: um deles entra primeiro, baixa o saldo e o outro encontra o saldo atualizado e recebe `EstoqueInsuficiente`. O Monitor, que tem 1 unidade, é o exemplo na demo.

? Como a NotificacaoService simula falhas? (parte sua)

Ela tem um contador `falhasSimuladas` (`AtomicInteger`). O método `simularFalhas(n)` define quantas falhas virão. A cada tentativa de envio, se o contador for maior que zero, ele diminui e a notificação lança `RuntimeException("servidor de e-mail indisponivel")`. No cenário 5 são 2 falhas (a 3ª tentativa funciona); no cenário 6 são 3 (vai para a DLQ).

? O que o App faz? (parte sua)

Cria o `EventBus` e os 5 serviços, mostra o menu, lê a opção e executa o cenário. Cada cenário cria pedidos pelo `PedidoService`, espera 3 a 4 segundos (`Log.pausa`) para o processamento assíncrono terminar e imprime o status final. O `0` chama `bus.shutdown()` e encerra.

05 Cassiano Abreu: casos de uso no mercado

SLIDES 10 E 11 | 2 MIN 30 DE 10 MIN | ANTES: WYLLIAN · DEPOIS: NAOKI

5.1 Seu papel e suas fronteiras

Você tira a parte do laboratório. Primeiro mostra que **EDA é usada de verdade** por empresas grandes; depois explica o que mudaria do projeto para a **produção**. É a parte que mostra maturidade técnica: o grupo sabe que o próprio projeto é uma simulação.

- **É seu:** quem usa eventos em produção (slide 10) e do projeto à produção, com a lição da Netflix (slide 11).
- **Não é seu:** definir EDA ou broker (Enzo), desenhar a arquitetura (Caio), o passo a passo (Wyllian).
- **Mas é seu:** você escreveu `PagamentoService` e `EstoqueService`. Perguntas sobre limite de R$ 5.000, estorno e idempotência caem em você; veja 1.2, 2.6 e 4.6.

5.2 Roteiro de fala

Tempo	Slide	Fala
0:00 a 0:10	RECEBE A PALAVRA	Obrigado, Wyllian. Até aqui vimos o projeto no laboratório. Agora, quem usa eventos em produção.
0:10 a 1:10	SLIDE 10 · QUEM USA EVENTOS	A Netflix é um dos exemplos mais citados: em apresentações da própria empresa, centenas de bilhões de eventos por dia passam pelo Kafka. A Uber chegou a atender mais de cem casos de uso em uma única plataforma de eventos. O Spotify usa Kafka Streams para playlists e recomendações em tempo real. E o Goldman Sachs usa Kafka e Flink em dados de mercado, trading e risco. São setores bem diferentes, mas o motivo é o mesmo: muita coisa acontece o tempo todo, e vários sistemas precisam reagir a isso sem depender uns dos outros. Esses números são de empresas em uma escala muito maior que a nossa, mas o princípio é o mesmo do projeto: um fato acontece, é publicado, e quem precisa reage.
1:10 a 2:20	SLIDE 11 · DO PROJETO À PRODUÇÃO	O nosso projeto é uma simulação. Hoje o EventBus roda em memória, na mesma JVM: se o programa encerra, tudo se perde, e os eventos que falharam ficam só na DLQ, que apenas os guarda. Em produção, o broker seria Kafka ou RabbitMQ, cada serviço rodaria no seu próprio processo, e teríamos banco de dados e uma DLQ que pode ser reprocessada. A ideia do código é a mesma; muda a infraestrutura. E a Netflix deixa uma lição: usar eventos onde queremos desacoplar, e orquestração onde o processo precisa ser observado. Ou seja, EDA é uma ferramenta poderosa, mas não é para tudo. Em produção, cada serviço teria o seu banco e o EventBus seria um cliente Kafka; os eventos e a lógica dos serviços continuariam praticamente iguais.
2:20 a 2:30	PASSAGEM	Com isso encerramos a parte teórica. O Naoki vai rodar o sistema ao vivo.

Contagem do roteiro: 284 palavras (ritmo calmo: cerca de 125 a 130 palavras por minuto).

> **CUIDADO**
> Os números de empresas têm **data**. Diga "centenas de bilhões" sem prometer um valor atual, e prefira "chegou a" e "em apresentações da própria empresa". Veja a verificação na seção 5.4.

5.3 Como conduzir os 2 slides

Slide	O que apontar	Cuidado
10 Quem usa eventos 0:10 a 1:10	Os quatro cartões em ordem: Netflix, Uber, Spotify, Goldman Sachs. Dê uma frase a cada um.	Não leia as fontes do rodapé. Se o professor pedir, cite: Svix, Confluent (Kafka Summit NYC 2017) e Software Patterns Lexicon.
11 Do projeto à produção 1:10 a 2:20	Primeiro a caixa "Hoje" (3 itens), depois a seta, depois "Produção" (3 itens) e por fim a faixa azul.	A faixa azul é a lição da Netflix: leia devagar. Não prometa que o projeto "já está pronto para produção".

> **COMPLEMENTO · DESCRIÇÃO TÉCNICA**
> O PDF `docs/Descricao Tecnica EDA.pdf` tem, na seção 4, exemplos genéricos de uso: e-commerce, bancos e fintechs, logística e IoT. Use como ponte se o professor pedir um exemplo que não seja empresa grande.

5.4 Estudo: os quatro casos e o que foi conferido

> **VERIFICAÇÃO EXTERNA**
> Os números abaixo **ainda precisam ser conferidos na fonte primária**. Eles vêm das conferências e fontes indicadas. Antes de citar um número em voz alta, confirme na fonte primária. Quando não achar a fonte, use a frase de "como falar" da tabela.

A tabela diz o que o slide afirma, o que a fonte indicada mostrou e como falar sem arriscar.

Empresa	O que o slide diz	O que a conferência mostrou	Como falar
Netflix (streaming)	Centenas de bilhões de eventos por dia no Kafka.	Indicado como dado de 2015 a 2017, em palestra da Netflix no Kafka Summit 2017, que mostrou 700 bilhões de eventos ingeridos por dia e mais de 1 trilhão processados. Fontes mais recentes (não primárias) falam em mais de 2 trilhões. Não usar o número recente sem fonte primária.	"Em apresentações da própria empresa, centenas de bilhões de eventos por dia, e o volume só cresceu."
Uber (mobilidade)	Mais de 100 casos de uso em tempo real em uma só plataforma de eventos.	Indicado como confirmado: palestra do time de Marketplace da Uber no Kafka Summit. A plataforma de processamento de eventos complexos está em produção há mais de 1 ano, com mais de 100 casos de uso em tempo real, usando Kafka e Samza.	"A Uber chegou a atender mais de cem casos de uso em uma plataforma de eventos."
Spotify (música)	Playlists e recomendações em tempo real com Kafka Streams.	Indicado só em fonte secundária (Software Patterns Lexicon). Não conferido em blog de engenharia ou palestra do Spotify.	"Segundo a fonte do slide…", sem detalhes técnicos extras.

Goldman Sachs (finanças)	Dados de mercado, trading e risco com Kafka e Flink.	Indicado só em fonte secundária (a mesma do slide). Não conferido em fonte primária.	Idem: "segundo a fonte do slide".

DECISÃO DO GRUPO · ANTES DA BANCA

Façam uma busca de 10 minutos por uma **fonte primária** (blog de engenharia ou palestra) do Spotify e do Goldman Sachs. Se não acharem, mantenham o "segundo a fonte do slide" e **não** acrescentem números.

ATENÇÃO

Esses quatro exemplos são mais de **streaming de eventos em larga escala** (Kafka como plataforma de dados) do que de **sagas de pedido**, como o nosso projeto. O ponto em comum é o **evento como unidade de comunicação**. Se perguntarem, diga isso: é honesto e mostra que vocês entendem a diferença.

5.5 Estudo: do projeto à produção

O slide 11 tem três itens de cada lado. Aqui está o mapa completo, peça por peça, para você responder "e o que muda de verdade?".

No projeto (hoje)	Em produção	Por quê
`EventBus` em memória	Kafka ou RabbitMQ	Broker externo, replicado: não é mais ponto único de falha.
Tudo na mesma JVM	Cada serviço em seu processo (container ou máquina)	Os serviços passam a falhar e escalar de forma independente; a comunicação vai pela rede.
`Event Store` em uma lista	Log persistente do broker (Kafka) ou event store em banco	Sobrevive a reinícios e permite **replay**.
DLQ que só guarda	Fila ou tópico de DLQ monitorado e reprocessável	Alguém é avisado e o evento pode ser reenviado depois de corrigir a causa.
Retry com `sleep` de 300 ms	Política de retry com espera crescente e limite	Não prende uma thread de trabalho enquanto espera.
Pool de 8 threads	Consumidores paralelos (grupos de consumidores, partições)	Escala horizontalmente.
`processados` (um `HashSet`)	Chave de idempotência gravada em banco	Sobrevive a reinícios e funciona com várias instâncias do serviço.
Status dos pedidos em um `Map`	Banco de dados de cada serviço	É o "banco de dados" do slide 11.
`Envelope` (id e horário)	Metadados da mensagem (id, horário, id de correlação)	Rastrear um pedido por todos os serviços.
Ordem não garantida	Chave de partição igual ao `pedidoId` (no Kafka)	Eventos do mesmo pedido ficam na mesma partição e mantêm a ordem.

PONTO FORTE

O que não muda: os eventos (continuam sendo fatos imutáveis), a lógica de cada serviço e a ideia de `subscribe` e `publish`. Trocar o `EventBus` por um cliente Kafka seria, no desenho, mudar o **meio**; o modelo continua o mesmo.

Kafka e RabbitMQ em uma tabela

	Kafka	RabbitMQ
Modelo	Log distribuído de eventos, dividido em partições	Broker de filas com *exchanges* e regras de roteamento
Retenção	Guarda os eventos por um período configurado; consumidores podem **reler** (replay)	Em geral, a mensagem sai da fila depois de confirmada pelo consumidor
Uso típico	Alto volume, streaming, vários consumidores lendo o mesmo histórico	Filas de trabalho, roteamento flexível, entrega mensagem a mensagem
DLQ	Por convenção (um tópico à parte)	Recurso nativo (*dead-letter exchange*)

Comparação resumida; ambos têm variações e exceções. Para a banca, basta a ideia: Kafka é um **log de eventos que se pode reler**; RabbitMQ é um **broker de filas** com roteamento.

5.6 Estudo: a lição da Netflix

A faixa azul do slide 11 diz: **"eventos onde desacoplar, orquestração onde o processo precisa ser observável"**. Em outras palavras, não é um ou outro: cada estilo resolve um problema.

	Coreografia (eventos)	Orquestração
Quem manda	Ninguém; cada serviço reage aos eventos	Um serviço central diz o próximo passo
Acoplamento	Baixo	Maior: o orquestrador conhece todos os passos
Ver o processo inteiro	Difícil: o fluxo está espalhado	Fácil: o orquestrador mostra em que passo cada processo está
Quando usar	Desacoplar, somar consumidores, reagir a fatos	Processos longos que precisam ser monitorados e ter erros tratados de forma explícita
No projeto	É o que fazemos (saga coreografada)	Seria um `PedidoOrquestrador` chamando Pagamento e depois Estoque

Conhecimento geral, **não está no slide**: do outro lado do espectro, a própria Netflix desenvolveu o Conductor, um motor de orquestração de fluxos de trabalho entre microsserviços. Use só se perguntarem por um exemplo de orquestração.

5.7 Perguntas que o professor pode fazer

? Por que não usaram Kafka no projeto?

O objetivo é demonstrar o **modelo**, e a atividade pede uma demo simples e funcional. Em Java puro, sem dependências, temos pub/sub, entrega assíncrona, retry, Event Store e DLQ. Subir Kafka exigiria infraestrutura que atrapalharia a demo ao vivo.

? Qual a diferença entre Kafka e RabbitMQ?

Kafka é um **log distribuído** em que os eventos ficam guardados e podem ser relidos; RabbitMQ é um **broker de filas** com roteamento flexível, em que a mensagem em geral sai da fila depois de confirmada. Ambos servem como broker de uma EDA.

? O que quebraria se trocássemos o EventBus por Kafka sem mudar mais nada?

O `publish` e o `subscribe` seriam trocados por clientes Kafka, mas aparecem problemas novos: os eventos precisam ser **serializados**; a rede traz falhas parciais; o `processados` do Estoque precisa ir para um banco; e a ordem só é mantida dentro da partição. Ou seja: a lógica dos serviços aguenta, mas a **robustez** (idempotência persistente, ordem, retry) precisa ser reforçada.

? Em produção, e se o serviço grava no banco mas cai antes de publicar o evento?

É o problema da **escrita dupla**. A solução comum é o padrão **Outbox**: o evento é gravado no mesmo banco, na mesma transação do dado, e um processo à parte o publica. O projeto não implementa isso, pois não há banco.

? EDA é a mesma coisa que Dataflow/Streams (outro tema da atividade)?

Se sobrepõem, mas não são iguais. EDA é reagir a **eventos discretos** (um pedido foi criado). Streaming de dados trata um **fluxo contínuo** com operadores (filtrar, agregar). Kafka e Flink são usados nos dois.

? A Netflix usa só eventos?

Não, e é isso que o slide 11 diz: eventos onde se quer desacoplar, orquestração onde o processo precisa ser observável. Misturar os dois é a prática comum.

06 Gabriel Naoki: demonstração ao vivo

SLIDES 9 E 12 | FORA DOS 10 MIN | ANTES: CASSIANO

6.1 Seu papel e suas fronteiras

Você é o **live preview**: metade da nota (50%) depende da demonstração funcionar e de o código seguir o modelo. Também é quem cuida do **repositório** e do **README**. Sua fala não conta nos 10 minutos de teoria, mas é a que a banca mais lembra.

- **É seu:** abrir a demo (slide 9), rodar os cenários, apresentar o repositório e encerrar (slide 12).
- **Não é seu:** reexplicar a teoria. A teoria já foi dita; a demo é **prova**. Cada cenário deve ligar a um conceito que a plateia acabou de ouvir.
- **Importante:** a demo roda em tempo real, sem rede. O que pode dar errado é ambiente (JDK, Maven, terminal), não código. Veja o plano B em 6.2.

6.2 Preparação e plano B

Comandos

```
# 1) compilar e gerar o jar (precisa de JDK 21 e Maven; faça isto com internet, antes)
mvn clean package

# 2) executar
java -jar target/eda-seminario.jar

# alternativa sem gerar o jar
mvn compile exec:java
```

ATENÇÃO · MAVEN

O comando `mvn clean package` **não pôde ser executado** no ambiente de preparação deste manual (sem acesso ao repositório do Maven). O código foi compilado e executado com `javac` e `java` (JDK 21), sem dependências. Por isso, **rode o Maven na sua máquina antes do ensaio**. Se o Maven falhar, use o plano B abaixo.

Checklist

Quando	O que conferir
Um dia antes	`java -version` mostra 21 ou superior. `mvn -v` funciona. Rodar `mvn clean package` **com internet** (o Maven baixa os plugins na primeira vez). Confirmar que existe `target/eda-seminario.jar`. Rodar os 8 cenários uma vez.
Antes de entrar	Levar o `.jar` já gerado (no pen drive e na nuvem). Testar o computador no projetor. Aumentar a fonte do terminal (Ctrl e +) e usar fundo escuro. Fechar notificações.
Antes de começar	Abrir o terminal na pasta do projeto, com `java -jar target/eda-seminario.jar` já digitado ou no histórico. Rodar uma vez e sair com `0`, para o processo estar "quente", e rodar de novo para zerar o estado.

Durante	Digitar o número e Enter. Esperar a mensagem `>> Status final` antes do próximo cenário (cada um leva 3 a 4 segundos). Deixar o **8 por último**.

Se algo falhar

Problema	Solução
`mvn` não instalado ou sem internet	Usar o `.jar` pronto: `java -jar target/eda-seminario.jar`. É o motivo de levá-lo gerado.
Erro de versão (`UnsupportedClassVersionError` ou "release 21 not supported")	O JDK é anterior ao 21. Rodar `java -version` e instalar o JDK 21.
Sem Maven, mas com JDK instalado (plano B)	Compilar direto: `javac -d out $(find src -name '*.java')` e depois `java -cp out eda.App` (no Linux e no macOS). Esta é a forma testada na preparação deste manual.
O programa travou ou o estado ficou confuso	`Ctrl+C` e abrir de novo: tudo é em memória, então reiniciar **zera** estoque, numeração e Event Store.
Letras estranhas no terminal	As mensagens do programa não têm acento de propósito ("Opcao", "nao"), para não depender da codificação do terminal.
A demo não roda de jeito nenhum	Mostrar a saída do cenário 3 que já está no **slide 8** e narrar o fluxo. Ter também um print ou gravação de tela dos 6 cenários principais.

> **CUIDADO**
> O Maven só consegue compilar offline se já tiver baixado os plugins pelo menos uma vez. **Não deixe o primeiro `mvn clean package` para o dia da apresentação.**

6.3 Roteiro de fala e de execução

Sugestão de demo curta, para uns 5 a 6 minutos. Cada passo liga a um conceito da teoria. O cenário 2 e o cenário 5 ficam de reserva se sobrar tempo ou se a banca pedir.

Abertura (slide 9)

Tempo	Fala
0:00	OBRIGADO, CASSIANO. ATÉ AGORA VIMOS O MODELO. AGORA VOU MOSTRAR O MESMO SISTEMA RODANDO. ELE É FEITO EM JAVA 21 PURO, SEM NENHUMA BIBLIOTECA EXTERNA. VOU USAR O JAR E USAR O MENU. EM CADA CENÁRIO VAMOS VER, NO LOG, O HORÁRIO, A THREAD E O SERVIÇO QUE REAGIU, E A IDEIA É QUE VOCÊS RECONHEÇAM O QUE O ENZO, O CAIO E O WYLLIAN EXPLICARAM.

Contagem do roteiro: 66 palavras (ritmo calmo: cerca de 125 a 130 palavras por minuto).

Sequência de cenários

Ordem	Cenário	O que dizer (uma ou duas frases)	O que apontar no log
1	1 Pedido com sucesso	"Este é o caminho feliz: PedidoCriado, pagamento aprovado, estoque reservado, pedido CONFIRMADO e e-mail."	Os 4 serviços reagindo e `PED-1 -> CONFIRMADO`. A coluna `[main]` só na 1ª linha.

2	3 Sem estoque (saga)	"Agora pedimos 3 monitores e só existe 1. O estoque avisa, o pedido é cancelado e o pagamento é estornado, sem ninguém coordenar."	`SEM ESTOQUE`, `CANCELADO (sem estoque)` e a linha `ESTORNO`. É a mesma do slide 8.
3	4 3 pedidos ao mesmo tempo	"Três pedidos seguidos. Repare que o status logo depois do publish ainda é CRIADO: a consistência é eventual. Depois todos convergem."	A linha `CRIADO / CRIADO / CRIADO` e depois `CONFIRMADO` x3; threads diferentes.
4	6 Falha permanente (DLQ)	"A notificação vai falhar de propósito. O bus tenta 3 vezes e manda o evento para a Dead Letter Queue. O pedido continua confirmado: a falha de um consumidor não derruba os outros."	As três linhas `tentativa n/3`, depois `DLQ <-`, e `Mensagens na DLQ: 1`.
5	7 Evento duplicado	"Reenviamos o mesmo PagamentoAprovado. O estoque reconhece e ignora: é o receptor idempotente."	`evento DUPLICADO ignorado (idempotencia)`. A Auditoria registra o duplicado, e só o Estoque o ignora.
6	8 Event Store e DLQ	"Por fim, o histórico de tudo que foi publicado, com id, horário e tipo, e o que ficou na DLQ."	A lista de eventos e a `DEAD LETTER QUEUE (1)`. Rode por último.

Encerramento (slide 12)

Tempo	Fala
0:00	PARA FECHAR: O CÓDIGO ESTÁ NO GITHUB E TODOS OS CINCO INTEGRANTES TÊM COMMITS. O CAIO FEZ O NÚCLEO DO BARRAMENTO, O CASSIANO, PAGAMENTO E ESTOQUE, O ENZO, EVENTOS, PEDIDOS E AUDITORIA, O WYLLIAN, NOTIFICAÇÃO E A DEMONSTRAÇÃO, E EU CUIDEI DO REPOSITÓRIO E DA DOCUMENTAÇÃO. OBRIGADO! FICAMOS À DISPOSIÇÃO PARA PERGUNTAS.

DICA

Rode o cenário 3 antes do 1 se quiser que a tela mostre `PED-1` igual ao slide 8: a numeração `PED-n` continua crescendo enquanto o programa está aberto. Se rodar o 1 primeiro, o cenário 3 mostrará `PED-2`, e tudo bem: só avise a plateia.

CUIDADO

Não explique a teoria de novo durante a demo e **não fique em silêncio** durante os 3 a 4 segundos de espera. Use a espera para dizer o que vão ver: "estão acontecendo, em paralelo, o pagamento, o estoque e a auditoria".

6.4 Os 8 cenários e as saídas esperadas

#	Cenário	O que prova	Resultado esperado
1	Pedido com sucesso (Ana, 2x Mouse, R$ 150)	Fluxo completo da saga	`CONFIRMADO` e e-mail enviado
2	Pagamento recusado (Bruno, Notebook, R$ 9.000)	Falha no 1º passo; nada a compensar	`CANCELADO (pagamento recusado)`
3	Sem estoque (Carla, 3x Monitor, R$ 3.000)	Compensação: estorno (saga coreografada)	`CANCELADO (sem estoque)` + `ESTORNO`

4	3 pedidos ao mesmo tempo	Assincronia e consistência eventual	`CRIADO` x3, depois `CONFIRMADO` x3
5	Falha temporária na notificação	Retry: falha 2x, funciona na 3ª	`CONFIRMADO`; e-mail enviado na 3ª tentativa
6	Falha permanente na notificação	Dead Letter Queue	`CONFIRMADO`; DLQ com 1 mensagem
7	Evento duplicado	Idempotência	Estoque ignora o duplicado
8	Ver Event Store e DLQ	Histórico de eventos e DLQ	Lista de eventos e da DLQ (rode por último)

As saídas dos cenários 1, 4, 5, 6 e 7 abaixo foram obtidas numa execução de referência (JDK 21, sequência 1 a 8). Os ids e horários do cenário 8 variam a cada execução. Horários e números de `worker` mudam a cada execução; o que importa é a **sequência**. As linhas `[AUDIT]` foram omitidas para economizar espaço (elas aparecem na tela).

Cenário 1: sucesso

```
PedidoService        PED-1 criado (Ana, 2x Mouse, R$ 150.00)       [main]
PagamentoService     PED-1 pagamento APROVADO                       [worker-1]
EstoqueService       PED-1 reservado 2x Mouse (restam 48)           [worker-3]
PedidoService        PED-1 -> CONFIRMADO                            [worker-5]
NotificacaoService   e-mail para Ana: pedido PED-1 confirmado!      [worker-6]
>> Status final de PED-1: CONFIRMADO
```

Cenário 4: 3 pedidos ao mesmo tempo

```
PedidoService  PED-4 criado (Ana, 1x Mouse, R$ 80.00)    [main]
PedidoService  PED-5 criado (Bia, 1x Mouse, R$ 90.00)    [main]
PedidoService  PED-6 criado (Caio, 3x Mouse, R$ 200.00)  [main]
>> Os 3 publish() ja retornaram. Status agora: CRIADO / CRIADO / CRIADO  (consistencia eventual)
EstoqueService PED-4 reservado 1x Mouse (restam 47)
EstoqueService PED-6 reservado 3x Mouse (restam 44)
EstoqueService PED-5 reservado 1x Mouse (restam 43)
>> Status final: CONFIRMADO / CONFIRMADO / CONFIRMADO
```

Cenário 5: retry (falha 2x, funciona na 3ª)

```
[worker-1] EventBus  NotificacaoService falhou em EstoqueReservado (tentativa 1/3): servidor de e-mail
indisponivel
[worker-1] EventBus  NotificacaoService falhou em EstoqueReservado (tentativa 2/3): servidor de e-mail
indisponivel
[worker-1] NotificacaoService  e-mail para Ana: pedido PED-7 confirmado!
>> Status final de PED-7: CONFIRMADO (a notificacao falhou 2x e deu certo na 3a tentativa)
```

Cenário 6: Dead Letter Queue

```
[worker-3] EventBus  NotificacaoService falhou em EstoqueReservado (tentativa 1/3): servidor de e-mail
indisponivel
[worker-3] EventBus  NotificacaoService falhou em EstoqueReservado (tentativa 2/3): servidor de e-mail
indisponivel
[worker-3] EventBus  NotificacaoService falhou em EstoqueReservado (tentativa 3/3): servidor de e-mail
indisponivel
[worker-3] EventBus  DLQ <- EstoqueReservado [PED-8] destino: NotificacaoService
>> Status final de PED-8: CONFIRMADO (pedido nao foi afetado pela falha da notificacao)
>> Mensagens na DLQ: 1
```

Cenário 7: evento duplicado

```
>> Reenviando PagamentoAprovado de PED-9 (simula entrega duplicada)
[worker-3] EstoqueService  PED-9 evento DUPLICADO ignorado (idempotencia)
```

Repare que a Auditoria registra o duplicado (ela assina tudo) e **só o Estoque o ignora**. A idempotência está no consumidor, que precisa dela.

Cenário 8: Event Store e DLQ (exemplo depois do cenário 6)

```
--- EVENT STORE (3 eventos) ---
d738fd5f | 02:05:18.444 | PedidoCriado | PED-1
fca6ddea | 02:05:18.949 | PagamentoAprovado | PED-1
abb8262b | 02:05:19.363 | EstoqueReservado | PED-1
--- DEAD LETTER QUEUE (1) ---
abb8262b | EstoqueReservado | PED-1
```

O id da DLQ (`abb8262b`) é o mesmo do `EstoqueReservado` no Event Store: é o mesmo envelope.

6.5 O estado entre os cenários

Tudo é em memória e **não é reposto** entre os cenários. Se rodar em sequência, o estoque e a numeração mudam. Conhecer isso evita susto na frente da banca.

Cenário	Pedidos criados	Efeito no estoque	Eventos no Event Store (acumulado)
1	PED-1	Mouse 50 → 48	3
2	PED-2	nenhum (recusado antes do estoque)	5
3	PED-3	nenhum (Monitor 1, pediu 3)	8
4	PED-4, PED-5, PED-6	Mouse 48 → 43	17
5	PED-7	Mouse 43 → 42	20
6	PED-8	Mouse 42 → 41	23 (e DLQ com 1)
7	PED-9	Mouse 41 → 39 (o duplicado não baixa de novo)	27

Valores calculados a partir do código e conferidos com uma execução de 1 a 7 em sequência, com o oitavo cenário no fim. O Notebook (5) e o Monitor (1) nunca mudam nessa sequência. **Reiniciar o programa zera tudo.**

> **ATENÇÃO**
> O cenário 3 **sempre** falha por falta de estoque, mesmo repetido: o Monitor tem 1 unidade e o pedido é de 3. O cenário 2 **sempre** é recusado: R$ 9.000 passa do limite de R$ 5.000.

6.6 O repositório e as evidências de commits

A atividade exige que o repositório tenha **commits de todos os 5 integrantes** (e conta na nota). Os números abaixo foram contados do histórico do git (`git log --all`).

Integrante	Nome(s) do autor no git	Commits sem merge	O que entregou

Gabriel Naoki	Gabriel Naoki (6); Gabriel Naoki Uto Turigoe (1)	7	Estrutura inicial (`pom.xml`, `.gitignore`, pastas); README; documentação (inclui a pasta `docs/` com os 2 PDFs)
Caio Abreu	Caio4breu (2)	2	`Event`, `Envelope`, `Log` e `EventBus` (pub/sub assíncrono, retry e DLQ)
Enzo Aquino	Enzo Aquino (3)	3	Eventos de domínio, `PedidoService` e `AuditoriaService`
Cassiano Abreu	NomsCodes (2)	2	`PagamentoService` e `EstoqueService`
Wyllian Mariano	Wyllian Mariano (2)	2	`NotificacaoService` e `App` (menu com 8 cenários)

Total: **23 commits no histórico**, dos quais **7 são merges** e **16 são commits sem merge**. O trabalho foi feito em **7 pull requests mesclados**: #1 (Caio), #2 (Enzo), #3 (Cassiano), #4 (Wyllian) e #5, #6, #7 (Gabriel, todos de documentação). Alguns integrantes aparecem com mais de um nome (ex.: `Caio4breu`, `NomsCodes`, `EnzoAkino`) porque o nome do autor no git é configurado em cada computador.

> **CUIDADO**
> **Confira no GitHub antes de entregar:** em *Insights → Contributors*, devem aparecer **cinco pessoas**. O GitHub liga o commit a uma conta pelo **e-mail** do autor. Se algum e-mail do git não estiver cadastrado na conta, o commit aparece sem avatar e pode não contar como contribuição daquela pessoa. Também verifiquem se o repositório está acessível ao professor (público, ou privado com acesso concedido).

6.7 Perguntas que o professor pode fazer

? **O que, no código, mostra que é uma EDA?**

Os serviços **não se importam entre si**: cada um recebe o `EventBus` e faz `subscribe`/`publish`. A comunicação é por eventos imutáveis (`record`), entregues de forma assíncrona em outras threads, com retry e DLQ. Dá para provar no log: só a primeira linha é `main`; o resto é `worker-N`.

? **Como eu adiciono um serviço novo?**

Criando uma classe cujo construtor faz `subscribe`, e instanciando-a no `App`. **Nenhum outro serviço muda.**

Exemplo ilustrativo (não está no repositório): um serviço de fidelidade.

```
public class FidelidadeService {
    public FidelidadeService(EventBus bus) {
        bus.subscribe("FidelidadeService", EstoqueReservado.class,
                e -> Log.info("FidelidadeService", "+10 pontos para " + e.cliente()));
    }
}
// no App.main:   new FidelidadeService(bus);
```

? Por que uma demo no terminal e não testes automatizados?

A atividade pede um live preview simples e funcional. Os 8 cenários cobrem o caminho feliz e as falhas, e funcionam como testes de aceitação. O repositório não tem testes automatizados; assumir isso é melhor do que esconder.

? Como vocês testaram a idempotência?

Pelo cenário 7: republicamos o mesmo `PagamentoAprovado` e o Estoque ignora, porque o `pedidoId` já está no conjunto `processados`. Sem isso, o estoque seria baixado duas vezes.

? O que o README e o pom.xml entregam?

O `pom.xml` configura Java 21, sem dependências, e gera `target/eda-seminario.jar` com `eda.App` como classe principal. O README explica EDA, traz o diagrama em Mermaid, a estrutura de pastas, como executar e a tabela dos 8 cenários. A pasta `docs/` traz também os dois PDFs.

07 Banco de perguntas gerais

PARA TODOS LEIA ANTES DA BANCA

7.1 Como usar esta parte

As perguntas específicas de cada parte estão no final de cada parte pessoal. Aqui estão as **transversais**, que podem cair em qualquer um e as mais difíceis. A etiqueta laranja indica **quem tem a resposta mais forte**, mas todos devem saber dar a resposta curta.

7.2 Conceituais

? Isso é realmente um sistema distribuído, se tudo roda em uma JVM?

[Cassiano / Caio]Honestamente:
o transporte é simulado
, e o grupo diz isso (README e slide 11). O que o projeto demonstra de verdade: serviços que não se conhecem, comunicação por eventos, assincronia, falha parcial, retry, DLQ e compensação. Para distribuir de verdade, troca-se o EventBus por Kafka ou RabbitMQ e cada serviço vira um processo. Como nenhum serviço importa outro, a lógica não muda.

? Qual a diferença entre EDA, Pub/Sub e Event Sourcing?

[Enzo]EDA é o estilo (reagir a eventos). Pub/Sub é o mecanismo de comunicação que usamos.
Event Sourcing
é outra coisa: guardar os eventos como a
fonte da verdade
do estado e reconstruir o estado por replay. O nosso Event Store é só um histórico de leitura, sem replay, então
não
é event sourcing.

? O sistema garante entrega "exatamente uma vez"?

[Caio]Não. Com retry, a semântica é de
pelo menos uma vez
: uma entrega pode se repetir. Por isso o consumidor precisa ser
idempotente
. "Exatamente uma vez" na prática é obtido com broker e consumidores idempotentes, não só com o broker.

? Como vocês garantem a ordem dos eventos?

[Wyllian]Não garantimos entre consumidores diferentes: eles rodam em paralelo (no cenário 4, o PED-3 reservou antes do PED-2). A ordem
dentro de um pedido
vem da causalidade: cada evento só é publicado depois do anterior. Em produção, no Kafka, usa-se o `pedidoId` como chave de partição.

? Quem coordena a saga?

[Enzo]Ninguém. É uma saga **coreografada**: cada serviço reage aos eventos dos outros. Na orquestração haveria um serviço central mandando cada passo; a vantagem é o desacoplamento, e o custo é não haver um lugar único que mostre o processo.

7.3 Sobre o código

? O que acontece se mil eventos forem publicados de uma vez?

[Caio]O pool tem 8 threads e uma fila interna de tarefas; os eventos esperam a vez. Como o retry usa `sleep` dentro da thread, um consumidor lento pode ocupar workers e aumentar a espera de todos. É aceitável na demo e é um dos motivos de usar um broker de verdade em produção.

? Por que as threads do pool são daemon?

[Caio]Para que o programa possa encerrar mesmo com o pool vivo: threads daemon não impedem o fim da JVM. O `App` também chama `bus.shutdown()` ao sair.

? Por que eventos são `record`?

[Enzo]Porque um evento é um fato imutável. O `record` gera construtor, acessores, `equals` e `toString`, e não tem como alterar os campos depois de criado.

? Os eventos carregam dados demais? Por que não só o id do pedido?

[Enzo]Carregar os dados (Event-carried state transfer) evita que o consumidor consulte o serviço de origem e preserva o desacoplamento. O custo é um contrato maior: se o formato muda, todos os consumidores que o usam precisam acompanhar. Há um meio-termo: o evento enxuto e o consumidor consulta quando precisar.

? Como vocês acompanham um pedido pelo sistema?

[Wyllian]Pelo log, que traz hora, thread e serviço, e pelo `pedidoId`, que todo evento carrega (`Event.pedidoId()`) e funciona como **id de correlação**. O Event Store também lista todos os eventos com id e horário.

? E se o `PedidoService` cair depois de criar o pedido e antes de publicar?

[Cassiano]O projeto não trata isso: o pedido existe no mapa sem evento. Em produção, usa-se o padrão **Outbox** (gravar o evento junto com o dado, na mesma transação, e publicar depois). Dizer isso é melhor do que fingir que não existe.

7.4 Sobre a apresentação

? Quem fez o quê no grupo?

[Naoki]Caio: núcleo do barramento de eventos (`Event`, `Envelope`, `Log`, `EventBus`). Cassiano: pagamento e estoque. Enzo: eventos, pedidos e auditoria. Gabriel Naoki: repositório e documentação. Wyllian: notificação e demonstração. É o que está no slide 12.

? Se tivessem mais tempo, o que melhorariam?

[Cassiano]Um broker real (Kafka ou RabbitMQ), serviços em processos separados, persistência para Event Store e estado dos pedidos, DLQ reprocessável, idempotência em todos os consumidores, testes automatizados e tracing por `pedidoId`. É o que o slide 11 resume.

> **SE PERGUNTAREM**
>
> Se **não souberem** uma resposta: "não implementamos isso no projeto, mas em produção seria feito com..." costuma ser melhor do que inventar. Os limites conhecidos estão na seção 1.5.

A Apêndices

GLOSSÁRIO COLA CHECKLISTS

A Glossário

Termo	Significado
Assíncrono	Quem pede não espera o resultado. No projeto, `publish()` volta na hora e a entrega ocorre em outras threads.
At-least-once (pelo menos uma vez)	Garantia de que o evento chega, mas que pode chegar repetido. É o caso com retry.
Broker	Intermediário que recebe os eventos e os entrega aos interessados. No projeto, o `EventBus`.
Compensação	Passo que desfaz o efeito de um passo anterior quando algo falha. No projeto, o estorno.
Consistência eventual	O estado só fica correto depois de um instante, sem intervenção. Ex.: CRIADO que vira CONFIRMADO.
Consumidor (consumer)	Quem se inscreve em um tipo de evento e reage.
Coreografia / orquestração	Coordenar por eventos, sem centro / coordenar por um serviço central.
DLQ (Dead Letter Queue)	Fila dos eventos que falharam em todas as tentativas.
Envelope	Embrulho do evento com id e horário.
Evento	Fato que já aconteceu, imutável. Ex.: `PedidoCriado`.
Event Store	Registro, em ordem, dos eventos publicados. No projeto, só leitura (sem replay).
Idempotência	Aplicar a mesma operação várias vezes tem o mesmo efeito que aplicar uma só.
JVM	Máquina virtual do Java; é o processo em que tudo roda no projeto.
Kafka / RabbitMQ	Brokers usados em produção. Kafka: log de eventos que se pode reler; RabbitMQ: filas com roteamento.
Live preview	Demonstração ao vivo do sistema durante a apresentação. Metade da nota.
Mermaid	Linguagem de texto para desenhar diagramas. O README do repositório usa para o fluxo de eventos.
Outbox	Padrão que grava o evento na mesma transação do dado e o publica depois, para não perder evento se o serviço cair.
Plano B	Alternativa para a demo se o Maven ou o ambiente falhar: `javac` + `java`, ou o print/gravação dos cenários.
Produtor (producer)	Quem publica o evento.
Pub/Sub (publicar-assinar)	Quem produz só publica; quem tem interesse assina; o broker liga os dois.
Pull request (PR)	Pedido de integração de uma branch no branch principal. No projeto, 7 PRs mesclados.

Record	Tipo do Java para dados imutáveis; usado em todos os eventos.
Retry (retentativa)	Tentar entregar de novo depois de uma falha. No projeto, até 3 vezes.
Saga	Processo longo dividido em passos locais, com compensação se algum falhar.
Thread / pool	Linha de execução / conjunto fixo delas. O `EventBus` usa um pool de 8. Threads `worker-N` são do pool; `main` é a da aplicação.
Topic (tópico)	O "assunto" em que se assina. No projeto, a classe do evento.

B Cola de uma página: frases prontas

Imprima esta página. Cada linha é uma resposta de uma frase.

Tema	Frase pronta
EDA em uma frase	"Serviços não se chamam: publicam eventos e reagem a eles. Quem publica não sabe quem vai reagir."
Broker	"O intermediário que recebe os eventos de quem publica e entrega para quem se inscreveu. No nosso código é o EventBus."
Arquitetura	"Nenhuma seta liga um serviço a outro. Tudo passa pelo EventBus."
Três propriedades	"Desacoplamento, assincronia e extensibilidade."
Assíncrono	"O publish retorna na hora; a entrega acontece em outras threads, em um pool de 8."
Consistência eventual	"O pedido fica CRIADO por um instante e converge para CONFIRMADO sem ninguém intervir."
Retry	"Até 3 tentativas, com 300 milissegundos entre elas."
DLQ	"Onde ficam os eventos que falharam nas 3 tentativas; não trava os outros consumidores."
Idempotência	"O Estoque guarda os pedidos já tratados e ignora o evento repetido."
Saga	"Pedido, Pagamento e Estoque coordenados só por eventos; se falta estoque, o Pagamento estorna."
Coreografia x orquestração	"Na coreografia cada serviço reage aos outros; na orquestração há um coordenador central."
Limite do projeto	"É uma simulação em memória, na mesma JVM; em produção, Kafka ou RabbitMQ."
Lição da Netflix	"Eventos onde desacoplar, orquestração onde o processo precisa ser observável."
Se não souber	"Não implementamos isso, mas em produção seria feito com..."

Números	Valores
Pool / tentativas / cenários	8 threads · 3 tentativas · 8 cenários
Pausas	Pagamento 500 ms · Estoque 400 ms · E-mail 200 ms · Estorno e retry 300 ms
Regras	Aprovado até R$ 5.000 · Estoque inicial: Notebook 5, Mouse 50, Monitor 1

C Checklist final, por pessoa

Marque cada item.

☐	Todos	Quando
☐	Ler as Partes 0, 1 e 7 e o Apêndice B	antes do 1º ensaio
☐	Combinar as 3 decisões da seção 0.4 (slide 9, slides 1 e 12, passagens)	antes do 1º ensaio
☐	Ensaiar a apresentação completa, cronometrada, com os slides abertos	2 vezes
☐	Ensaiar as passagens de palavra, trocando a frase exata	no ensaio
☐	Conferir os slides em PDF no computador que será usado	véspera
☐	Chegar com o terminal aberto, jar gerado e plano B (cópia do jar e prints)	dia
☐	Conferir o slide 7 (correção do "PedidoService confirma e notifica") e o slide 8 (seta "->")	véspera

☐	Cada pessoa	Alvo
☐	**Enzo:** cronometrar a fala dos slides 1 a 3; dizer a frase "quem publica não sabe quem vai reagir"; uma frase por padrão	2:30
☐	**Caio:** ensaiar o slide 5 sempre na mesma ordem (centro, topo, base); decorar "nenhuma seta liga um serviço a outro" e as 3 palavras	2:30
☐	**Wyllian:** usar só 3 linhas do slide 8 (1, 5 e 9); saber explicar por que o estorno vem por último; corrigir o slide 7	2:30
☐	**Cassiano:** ensaiar com "chegou a" e "em apresentações da própria empresa"; buscar fonte primária de Spotify e Goldman (seção 5.4)	2:30
☐	**Naoki:** gerar o jar com internet; rodar os 8 cenários; levar o jar e prints; conferir Contributors no GitHub e o acesso do professor; anexar link e PDF no Classroom	demo

D Como este manual foi conferido

Conferência feita contra o repositório, em 30/10/2026:

- **Código:** todos os arquivos `.java`, o `pom.xml` e o README foram lidos, com números de linha. A `Descricao Tecnica EDA.pdf` foi lida por completo. O `Slides EDA.pdf` do repositório é idêntico ao entregue.
- **Execução:** o projeto foi compilado com `javac` (JDK 21) e os cenários 1 a 8 foram rodados em sequência. As contagens do Event Store (3, 5, 8, 17, 20, 23, 27) e o estoque de Mouse batem com a seção 6.5. **O comando `mvn clean package` não foi executado** nesta conferência (ver 6.2).
- **Histórico git:** 23 commits, 7 merges, 16 commits sem merge, 7 PRs.
- **Não conferido:** os números externos de Netflix, Uber, Spotify e Goldman Sachs (seção 5.4). Devem ser conferidos na fonte primária antes de citar.